

路线 3 知识卡片

用途:路线 3(城堡执行——网关收敛、FGA 授权、串联裁决、审计防篡改、供应链体检、harness 确认门禁复刻)的复习知识卡片 卡片总数:85 张(主线 0029—0041 共 13 篇 68 张;harness 复刻 6 篇 17 张) 生成日期:2026-09-05 碎片:知识卡片-碎片/(主线按五阶段流水线产出,harness-* 为轻流程)

0029 网关上岗 —— 知识卡片

MCP 网关 (ContextForge Gateway)

是什么: MCP 网关是挡在 AI Agent (自主调用工具的 AI 程序) 和 MCP server (工具提供方, MCP 即 Model Context Protocol——Agent 调用外部工具的开放协议) 之间的独立转发组件。所有工具调用都改从它走, 由它统一认人、查权限、留记录。ContextForge (IBM 的开源实现, PyPI 包 `mcp-contextforge-gateway`) 官方定位是三合一: gateway (转发) 加 registry (工具注册表, 登记有哪些工具可调) 加 proxy (统一对外端点), 而且不只收 MCP、还收 REST/gRPC 这类普通 API。好比给大楼设了前台: 人人走大门, 前台认得出你是谁、能不能进、来了都登记。实测它有两套互不通用的登录凭证体系 (凭证面[自造]): 看板 (浏览器里的 Web 管理界面) 登录走邮箱账号, API 调用走 HTTP Basic 密码 (随请求明文带用户名密码的老式认证)。

解决什么问题: 没有网关时, Agent 直连各个 MCP server, server 自己不知道连接者是谁、有没有权限、干了什么, 出了事无从追责。网关把"谁在调用工具"收拢到一个位置, 认证 (核验身份)、授权 (判定权限)、审计 (留存记录) 就都能在同一点上做。另有一个使用注意: Basic 这条老通道官方默认是关的 (`API_ALLOW_BASIC_AUTH=false`), API 侧推荐改用 JWT (带签名的登录令牌)。

我们的办法: 把网关当"部署对象"而非自己写的代码——给它独立的虚拟环境 (依赖互相隔离的目录) 和独立配置目录, 只听本机回环地址 (127.0.0.1, 只有本机能连), 实测浏览器一打开就是登录墙 (未登录时访问被重定向到的登录页), `curl /health` 返回 `{"status":"healthy"}`。工作目录是它的"户口"——官方数据库默认用相对路径的 sqlite 文件 (`sqlite:///./mcp.db`, sqlite 即单文件嵌入式数据库), 同一命令在哪个目录跑就读哪份配置、用哪个库, 独立目录让两边互不污染。

默认封锁管理面 (secure by default)

是什么: secure by default 指安全产品出厂就把管理面 (产品自带的管理后台和管理接口) 关死、要管理员显式打开的设计取向。它和 secure by design (在架构设计阶段就内置安全) 是相邻的两回事——一个管"出厂配置", 一个管"图纸怎么画"。ContextForge 正是这种默认: 不动配置时 Web 看板 (浏览器里的管理界面) 和管理 API 都不启用, 启动日志明写 `Admin API routes not mounted` 和 `Static files not mounted - UI disabled`; 还有更典型的 fail-closed (出错或无配置时默认拒绝、而非默认放行) 实例——允许列表 (allowlist, 放行名单) 空着时, 默认封锁全部跨网关路由。

解决什么问题: 管理面本身就是攻击面——登录口一旦暴露, 攻击者就多一个爆破口令、调用管理接口的入口。美国政府联合指南把这套理念的价值说成两条: 缩小攻击面, 并把安全配置的负担从运营者身上卸掉——"忘了关"这类人为失误从根上被删掉。

我们的办法: 本地要看板才在配置里显式打开 `MCPGATEWAY_UI_ENABLED=true` 和 `MCPGATEWAY_ADMIN_API_ENABLED=true` 再重启, 实测把开关改回 false 后登录墙 (未登录时被重定向到的登录页) 立刻消失。注意一个坑: 官方开发示例 .env (存环境变量的配置文件) 里这两个开关默认写 true (官方明说是为了让本地开箱即用), 真正"出厂关死"的是代码默认值。

init_secrets (网关密钥生成器)

是什么: init_secrets 是 ContextForge 自带的密钥生成脚本, 用密码学安全的随机源 (人猜不出的高质量随机数) 一次生成三把密钥, 官方要求首次启动前就得跑。网关缺了密钥会 fail fast (启动即报错退出), 官方原话说连本地开发环境也不许缺。三把分别是 `JWT_SECRET_KEY` (签发登录令牌的私章)、`AUTH_ENCRYPTION_SECRET` (加密存储用的钥匙)、`BASIC_AUTH_PASSWORD` (API 的 HTTP Basic 密码)。

解决什么问题: 网关没有私章就签不出登录令牌, 整个认证直接瘫。让密钥由脚本随机生成而非人手定, 弱密码和密钥被抄进代码库这两类事故就从流程上被排除。

我们的办法：用 `--patch-env .env`（存环境变量的配置文件）把生成值直接补写进网关配置（官方设计是替换 `__REPLACE_ME__` 占位符）。密钥文件靠仓库的 `.gitignore` 挡在版本库外；实测三把全是随机值，全程没人工设过密码。另记两条通用纪律（出自 OWASP）：密钥要定期轮换（`BASIC_AUTH_PASSWORD` 改配置重启即可换新）；`.env` 这类环境变量可能进日志或系统转储、属次优做法——生产环境应上 `vault`（专用密钥保险库）。

CSRF 双票机制（CSRF double-submit cookie）

是什么：CSRF 即跨站请求伪造（Cross-Site Request Forgery）——一种借已登录浏览器的凭证冒充用户发请求的攻击。防御侧的经典做法是 `double-submit cookie`（卡片里的“双票”[自造比喻]：服务端把一张票种进 `cookie`，又在表单里塞一张票，提交时两张对上才放行）。机制成立的根因是浏览器只让同源 JS 读 `cookie`（同源即协议、域名、端口都相同）——外部小站的攻击脚本摸不到第一张票，自然也抄不出第二张，所以改状态的动作（比如登录）必须证明请求来自网关自己的登录页。`ContextForge` 登录页用的就是这套：种一张 20 分钟有效的 CSRF 通行 `cookie`，它刻意不带 `HttpOnly`（禁止 JS 读取的标志）——JS 要读它回填请求，双票才玩得转。

解决什么问题：它解决的是外部站点借你已登录的浏览器伪造改状态请求（比如提交登录表单）这类攻击。但这套双票牵出一个浏览器行为坑：`cookie` 带 `Secure` 标志（承诺只在 HTTPS 里递）时，浏览器在 `http://` 上会直接丢掉它——Chrome 对 `localhost` 放行，Safari 不放行。实测 Safari 提交登录直接报 `CSRF token cookie missing`，本地 `http` 环境走不完登录。

我们的办法：本地教学环境在配置里关掉 `SECURE_COOKIES` 再登录，生产保持默认开启并上 HTTPS，实测关掉后 Safari 也能走完登录。通用纪律：OWASP 已把朴素双票标为 `discouraged`（不推荐），更稳的是签名版 `signed double-submit`——票据用 HMAC（带密钥的哈希签名）绑到用户会话上。

uv venv --python（钉 Python 版本建环境）

是什么：这是 `uv`（Astral 出品、Rust 写的 Python 虚拟环境与包管理器）钉死解释器（运行 Python 代码的程序本身）版本建虚拟环境的命令，写成 `uv venv --python 3.12`。它针对的坑是版本约束冲突：很多包在元数据里声明 `requires-python`（包自己声明的 Python 兼容范围），解释器不满足时包管理器会强制拒绝安装。`uv` 选解释器还有条官方规则——挑第一个兼容版本而不是最新版本，所以钉 3.12 就复用本机的 3.12，不会自作主张换 3.13；本机没有对应版本时它会自己下载。

解决什么问题：实测撞过一次真实的约束——`ContextForge` 声明要求 `Python ≥3.12 且 <3.14`，而系统 `python3` 是 3.14.7，直接装会被包管理器拒绝。加 `--python 3.12` 后，`uv` 找到本机已有的 CPython 3.12（Python 官方参考实现）直接复用，冲突就地化解。

我们的办法：给网关单开一个钉在 3.12 的虚拟环境当依赖隔离的家，装包也指定这个环境的解释器，实测秒级建好、与 Agent 自己的环境互不污染。另一种等价钉法是在项目里放 `.python-version` 文件（一行写个版本号）。

0030 三 server 挂网关 —— 知识卡片

FastMCP 传输参数（MCP transport）

是什么：`transport` 是 `FastMCP`（把普通 Python 函数直接变成 MCP server 的框架）上决定 server 用什么通道收发消息的参数，写成 `mcp.run(transport=...)`。MCP（Model Context Protocol，Agent 调用外部工具的开放协议）规范说：传输只是绑定，管消息怎么封装送达，不管消息什么含义——所以同一份代码换传输不改业务逻辑。三种通道：`stdio` 是 server 被客户端当子进程拉起、经标准输入输出通信，每个会话各拉一个进程。`SSE`（Server-Sent Events，HTTP 长驻事件流）是 2024 年版协议的 HTTP 传输，官方措辞已 `deprecated`（弃用，由 `streamable-http` 取代）。它握手（协议握手：双方先互发几条问候消息确认彼此听得懂，再谈正事）的第一步是任何人 `/sse` 发 `GET`，立刻收到一条 `event: endpoint` 事件加带会话号的回传地址——先领“见面礼”再开口说话。`streamable-http` 是 2025 年版协议的单端点流式传输，消息走 HTTP `POST`，回复按需开请求级 `SSE` 流。注意参数值按包分家：官方 `python-sdk` 写 `transport="streamable-http"`，`FastMCP v2` 写 `transport="http"`——同一件事两套字面量，查文档先认包名。

解决什么问题：同一套工具既要被本机直连、又要以 HTTP 形态挂给网关（MCP 网关：代理并管理多个 MCP server 工具调用的中间层）时，不用写两份代码，换一个参数即可。它也把一个选型风险摆上台面：`SSE` 与 `streamable-http` 两套 HTTP 口味并不互通，两端吃的不一样就握手失败。`FastMCP` 官方建议新项目一律用 HTTP 传输，但这只是默认建议——MCP 规范自己的兼容思路是先试新传输、失败再回退旧传输，所以选型的现实依据是对

接的另一端实测吃哪个。实测踩过一次真实的坑：用 `streamable-http` 向网关注册，网关握手第一步是对 `/mcp` 发 GET 开长驻事件流。而 server 处于 `stateless` 模式（不为连接保留会话状态），收到 GET 就把会话终结掉，日志里三连 `Terminating session: None`，网关傻等 30 秒超时；换 SSE 一次通过。教训：看 server 日志里的请求路径加会话终结记录，十分钟就能定位协议不合，比查文档快。

我们的办法：三个自写 server 在启动处用 `MCP_TRANSPORT` 环境变量切换传输，默认仍是 `stdio`，挂网关时切到 SSE。

登记即盘点 (registration-time tool inventory)

是什么：「登记即盘点」[自造]指 MCP 网关登记上游时不只把名字记进表，而是亲自以 MCP 客户端身份对那个 URL 做一次完整握手（initialize 握手后 `tools/list` 抄工具清单）。这里的 MCP 网关即 ContextForge（IBM 的开源 AI 网关，把工具、Agent 和 API 汇成一条干净端点的注册与代理层）。这个动作在官方源码里就叫 `gateway initialization`：登记路径内跑一次远端握手，把抄回的工具列表直接入库。由此得到两个可用的性质：工具出现在网关工具列表里，就是那次真实握手成功的证明。上游 server 一挂，网关很快把它标成离线——感知机制不是事件推送，而是周期性健康检查（`periodic health check / heartbeat`，由主实例定时去探每个上游还活不活的心跳机制），最快也要等下一轮探测。配套机制是命名空间前缀：网关把工具改名为「server 名-工具名」，两个 server 即使有同名工具也不会撞车。

解决什么问题：网关要靠一张可信的工具总表来做可见性控制、团队范围限定和成员资格校验（ContextForge 官方原话是 `visibility + team scoping + membership validation`，即谁能看到哪些工具、令牌的团队归属是否匹配），并靠命名空间前缀避免撞车——例如 `read_file` 进了 `filesystem` 这个上游就变成 `filesystem-read-file`。若登记只存 URL 和名字，表里就会躺着一批实际调不通的死工具，授权决策建立在假清单上。上游死活的感知同样重要——网关要接管全部工具调用，前提是它知道每个上游此刻活不活着。

我们的办法：用 `POST /admin/gateways` 登记（body 只有 `name`、`url`、`description` 三件套），登记完立刻查网关工具列表就能看到全部带前缀的工具。实测杀掉一个上游 server 进程后，网关侧显示它离线而登记仍保留，重启进程即恢复。

JWT teams claim 的权限边界 (teams claim semantics)

是什么：teams claim 是 JWT（JSON Web Token，带签名的登录令牌）里名为 `teams` 的字段，在 ContextForge 网关（IBM 的开源 MCP 网关：把多个工具 server 代理成一条端点的中间层）里它决定令牌的权限边界，三种形态的语义完全不同。键缺席等于公开级（只能调用公开工具的最低权限档）——网关源码注释称之为 `secure default`（安全默认：宁可降权，也不默认放开）。空列表 `[]` 也等于公开级——它不是「什么都没有」，而是「明示只看公开内容」。只有 `teams: null` 且同时带 `is_admin`（令牌里的管理员标志字段）标志才是管理员旁路；反过来 `teams: null` 但 `is_admin=false` 仍被压回公开级，旁路不生效。按 RFC 7519（JWT 的标准文档）的命名分类，teams 属于 Private Claim（私有 claim：字段名不进标准注册表，语义完全由签发方和校验方自行约定，易撞名、易随版本漂移）。所以「给不给键、给空还是给 null」这三道门长什么样，全由实现方说了算，凭直觉猜必错。

解决什么问题：给网关签测试令牌时，若凭直觉以为「空列表等于无约束、等于最大权限」，实际拿到的只是公开级，基于它的权限测试会全盘得出错误结论。通用纪律：JWT 里任何用作权限边界的私有 claim，「缺席 / 空集合 / null」三种形态必须逐个核对语义；而且约定随时可能变——网关新版本起会话令牌（人登录会话用的令牌）已不再内嵌 teams claim（改为每个请求查数据库解析），只有 API 令牌（程序调接口用的令牌）仍内嵌，复习时记着这是特定版本的行为。

我们的办法：签发测试令牌用网关自带的 `create_jwt_token` 函数（60 分钟有效），要管理员旁路就显式传 `teams=None`，令牌里才会序列化出 `"teams": null`；实测这样的令牌能走通管理员接口，空列表令牌则被压到公开级。

毒注册打内网 (SSRF via poisoned gateway registration)

是什么：SSRF（Server-Side Request Forgery，服务端请求伪造）按 OWASP（发布权威攻防指南的安全行业组织）的定义，是攻击者滥用服务器自身的功能去读取或改写内网资源的攻击——不只是探测，还包括改写。在 MCP 网关（代理并管理多个 MCP server 工具调用的中间层）场景下，「毒注册」[自造]指攻击者向上游（被网关代理、以 URL 挂进来的 MCP server）注册接口提交一个指向内网的 URL，诱骗网关替他去连。成因正是 OWASP 那句：服

务端拉取远端资源，却不校验用户提供的 URL。毒从哪进：登记接口本身。网关收到注册请求后会亲自连到提交的 URL 去握手抄工具清单——这个「网关替你出手连网」的动作，正好被攻击者借作跳板（借别人机器的网络位置代发请求的中转点）。

怎么得手：拿到注册权限的攻击者提交形如 `http://10.0.0.5:6379/` 的内网地址，网关就会向它发握手请求；攻击者拿不到响应内容，只能从响应快慢和报错差异推断内网哪台机器活着、哪个端口开着——这有现成术语：blind SSRF（盲 SSRF：响应不回传给攻击者，只能靠这类侧信推断结果的探测形态）。

我们的对策：网关在注册入口默认拒收 localhost 和私网地址，报错明说被 SSRF protection 拦下，本地教学环境可用 `SSRF_ALLOW_LOCALHOST=true` 只放行回环地址（127.0.0.1，只有本机能连）。但要诚实标注成色：按地址黑名单拦截属 deny-list（deny-list：只拒已知坏地址的黑名单式防御），OWASP 明说 deny-list 天然可绕，DNS rebinding（DNS 重绑定：让同一域名先后解析到内外网不同 IP、绕过地址校验的攻击）、重定向、十进制 IP 等变体都能换着法绕开，更稳的是 allow-list（只放行显式许可地址的白名单）——所以它是必要但不充分的防线，和沙箱默认封私网出网同属「把内网不可达做成默认姿态」这一条设计思想。

0031 Agent 改走网关 —— 知识卡片

唯一入口收敛（single ingress convergence）

是什么：唯一入口收敛是一种把 Agent 的全部工具调用改到只经网关（挡在 Agent 和工具 server 之间、统一转发并管认证的中间层）一个入口、并拔掉所有绕行直连的防御改造。门建好了没人走等于没修。MCP（Model Context Protocol，Agent 调用外部工具的开放协议）官方安全文档把“令牌直通”（server 不验令牌就往下转发）定为 anti-pattern（公认有害的反面写法）。理由正是控制可被绕过、审计无法归因——门一旦被绕开，门上的闸就形同虚设，这跟 Agent 走直连是同一类病。授权、审计、令牌这类控制都装在门上的闸[自造]（挡在执行路径上的裁决关卡）里。这套比喻有标准对应物：NIST 零信任架构（零信任：默认不信任任何网络位置、每次访问单独验证）把同样的东西叫 PEP（Policy Enforcement Point，策略执行点——所有访问必须穿过的裁决关卡）。

解决什么问题：收敛是后续一切治理的前提——只有所有调用都过同一扇门，门上的闸才拦得住人。它还把“拓扑知识”（哪个 server 在哪、怎么启动）从 Agent 手里收走：官方文档警告 stdio（把 server 当子进程拉起、经标准输入输出通信的本地传输）模式下 server 可直接触达用户系统，收走启动知识等于收走攻击面。

我们的办法：把 Agent 的连接配置从“三个 stdio 子进程各一条”换成“网关一个 HTTP 端点”。实测一轮真实调用里，模型生成的就是网关目录里的名字 `filesystem-list-dir({"path": "."})`，拿回文件列表，全程没有任何直连子进程参与。

工具名即路由键（tool name as routing key）

是什么：指 Agent 眼里的工具名不只是个显示名，而是网关工具目录里的路由键——网关凭这个名字查表，决定把调用转发给哪个上游 server（被网关代理的工具提供方）。名字由网关的命名空间规则（给每个名字划分归属区域的规则）决定：登记时给每个工具加“server 名-工具名”前缀，比如 `read_file` 进了 filesystem 这个上游就变成 `filesystem-read-file`。

解决什么问题：入口收敛（所有工具调用只经网关一个入口、绕行直连全部拔掉）之后，Agent 从出生就只认识网关目录里的名字，带前缀的全名成了对外唯一契约。这带来一个连锁反应：名字一变，所有引用它的配置都得跟着换命名空间——例如审计里按工具名给数据分级的配置，写的也是全名，改名等于一次全局重命名。

我们的办法：把带前缀的全名当唯一对外名字，Agent 侧配置与审计分级配置一律按全名书写。实测模型生成的调用名本身就是全名，模型世界里不存在不带前缀的旧名。

短时通行证铸发（short-lived token minting）

是什么：短时通行证铸发是一种令牌供给形态：程序每次启动时才现铸一枚短时有效的 JWT（JSON Web Token，带签名的登录令牌），经环境变量注入进程，随进程生灭。铸币权——签发令牌用的私钥——始终留在网关一侧，这正符合非对称签名（私钥签发、公钥验签）的标准分工：消费者（拿令牌调工具的一方）手里永远只有票（即令牌本身）、没有铸票机。

解决什么问题：它把凭证泄露的损害窗口从“永久”压到“有效期之内”，与 OWASP（开放 Web 应用安全项目，应用安全领域的权威组织）的建议同构——用短有效期缓解令牌被盗后重放（拿偷来的票冒充原主再次使用）。但短有效期只压窗口、不等于吊销能力（吊销：在有效期前作废令牌）：JWT 是无状态令牌，过期前没法单方面作废，60 分钟

的票被抄走后的 60 分钟内仍是有效凭证，真吊销要另行机制。

我们的办法：独立的铸票脚本在 Agent 启动命令里现铸一枚 60 分钟的票并注入环境变量，令牌全程不落盘。实测把有效期改成约 1 秒、等 2 秒再拿它连网关，直接被 401（未认证，请求被拒）挡下——“短时”是可以亲手摸到的行为。

零插值 heredoc (quoted heredoc, <<'EOF')

是什么：heredoc (here document) 是 shell 里用 <<定界符 把一段多行文本原样喂给命令的写法。零插值 heredoc 是它在定界符（标记多行文本起止的字）任何部分带引号时的形态——POSIX (Unix 系统接口的官方标准) 规则：定界符任何部分带引号即整段不展开，所以 <<'EOF' 和 <<"EOF" 都算数，内容里的 \$、引号、空格、换行一律按字面传递。

解决什么问题：它避开 shell 引号嵌套这个经典坑。往命令替换 \$(...) (把命令的输出填回原位置的写法) 的双引号里嵌多行代码时，代码里的单引号会被外层引号环境撕碎——实测一段内嵌 Python 片段收到的是 user_data='email': ... 这样的 SyntaxError (语法错误)，引号被剥掉、参数被按空白切散。这种多层嵌套的碎裂没有单独的标准条目，只能靠实测确认。

我们的办法：把这段多行逻辑抽成独立脚本，脚本内用 <<'EOF' 把 Python 片段喂给解释器。实测字典参数原样到达，报错消失。

TS 官方 SDK 最小客户端 (@modelcontextprotocol/sdk)

是什么：这是 MCP (Model Context Protocol, Agent 调用外部工具的开放协议) 官方 TypeScript SDK (TypeScript 是微软出品的 JavaScript 超集语言) 提供的客户端拼装件。最小配方是 Client 类配 StreamableHTTPClientTransport (streamable HTTP 传输的客户端实现——按协议规范是单端点、消息一律走 HTTP POST)，几十行就是一个完整的 MCP 消费者。本卡写法锚定官方 v1.x 分支，仓库主线已到 v2。

解决什么问题：它让“统一入口不绑定某一种语言”变得可验证——第二个消费者用另一种语言、不依赖 Python 生态，走同一个网关门。如果这样也能连通并调到工具，说明收敛后的入口对所有 SDK 消费者（即调用网关工具的客户程序）一视同仁，而不是只为原先那个 Agent 定制的。

我们的办法：TS 侧在 requestInit.headers (HTTP 请求头配置，官方示例里带票的标准入口) 里带 Bearer 票 (Bearer 即“持票人”式认证，谁出示令牌就算谁)，依次 connect → listTools → callTool → close。实测列出全部 6 个带前缀工具并真实执行了一次调用。

双身份对照 (two-identity authorization test)

是什么：双身份对照是一种授权评估方法：给同一个入口准备两个权限不同的身份，对比它们各自实际能调到什么，用差异来检验权限边界是否真实存在。它不是自创动作，而是行业标准测试法——OWASP (开放 Web 应用安全项目) 的 WSTG (Web 安全测试指南) 里，越权测试 (拿低权身份去探高权资源，验证服务端拒绝) 就是这么打边界的。

为什么重要：只有一个身份时，访问控制矩阵 (记录“谁能在什么上做什么”的表格) 的身份维度无从验证——“受限身份”是不是真受限，没有参照物就说不清。更隐蔽的坑是单身份系统里“持票即有权”。而 MCP (Model Context Protocol, Agent 调用外部工具的开放协议) 官方安全文档明确：持有凭证不等于认证 (authentication, 验明“你是谁”)。两个身份一对照，票里缺身份粒度 (令牌里不含“这是谁、有什么权限”的信息) 这类缺陷才会现形。

我们怎么用：Python Agent 当运维位 (全权限身份)，TS 最小客户端预备只读位 (受限身份)，两者走同一个网关门。后续的越权测试就以只读位为靶，看网关对它放行了哪些、拒绝了哪些。

0032 OpenFGA 建模 —— 知识卡片

OpenFGA check 裁决接口 (OpenFGA check API)

是什么：check 是 OpenFGA 提供的授权裁决接口。OpenFGA 是 Google Zanzibar 论文的开源实现 (Zanzibar 是 Google 用关系元组做集中授权的系统)。输入是一个三元组 (user, relation, object) (谁、对什么对象、有什么关系)，响应体只有 {"allowed": true/false} 一种形态。语义是存在性判定：仅当存在一条直接或可递归到达的

关系路径（路径由 tuple 组成，tuple 是一条"谁对什么有什么关系"的记录）才返回 true，否则返回 false——等价于默认拒绝。check 在架构里扮演 PDP（Policy Decision Point，专职回答"允不允许"的集中式判定端点）。

解决什么问题：让"这个人能不能用这个工具"有唯一裁决点，而不是散落在各处业务代码里自己写 if——自己手写授权判断是安全实践里公认的灾难。存在性语义还省掉了"写拒绝名单"这门功课：用户 bob 对 shell 工具没有任何可到达的关系路径，check 自然返回 false，不需要任何人写过"拒绝 bob"。

我们的办法：用 docker 起一个内存存储的 OpenFGA，向 `/stores/{store_id}/check` 发 POST，body 带 `tuple_key` 和 `authorization_model_id`（即按哪版模型判）。跑六条 check 当回归基准。授权网关（拦下工具调用先问权限的中间层）是它唯一的调用方，check 返回 false 就被网关翻译成 HTTP 403。

四元组落成三元组（dimension as relation）

是什么：四元组落成三元组 [自造] 是把授权矩阵（把谁能用什么都列成一张表的业务侧写法）里"人 × Agent × 工具 × 资源"四个维度，翻译成 OpenFGA 只认的三元组 (user, relation, object) 的建模动作，规范等价说法是"把高维授权矩阵分解成关系图上的可达路径"。机制根基在 Zanzibar 的元组文法：user 端可以是一个 userset ("另一对象 #关系"形式的集合型用户)，所以 Agent 不必做成工具上的字段，而可以是工具对象上的一条关系。实例：给工具 `tool:shell-run-command` 写一条 `deployed_on`（部署于哪个 Agent）元组指向 `agent:demo`，"运维位通过这个 Agent 用 shell"就由模型规则推导出来，而不是存成一行配置。

解决什么问题：授权矩阵是业务语言，三元组是引擎语言，中间缺一条翻译规则就会退化成给每种"人-工具"组合硬写一条元组，矩阵每加一格数据就翻一倍。有了 userset 这条文法出路，给运维位上新工具只需新增一条 `deployed_on` 元组，人侧授权自动继承，不用挨个改人。

我们的办法：模型里建 user / agent / tool / resource 四个类型，工具类型上定义 `can_execute: [user] or admin from deployed_on`——人是 Agent 的 admin、工具部署在该 Agent 上，两条链一接，执行权就长出来了。

ReBAC 级联判定（ReBAC cascade evaluation）

是什么：级联判定是 ReBAC（Relationship-Based Access Control，基于关系的访问控制）引擎求值一条 check 时的走法：在关系图上找一条可达路径，而不是查一张静态表。落到 Zanzibar 的三个重写原语上：`_this`（直接查本对象的元组）、`computed_userset`（同一对象换一条关系继续查）、`tuple_to_userset`（先沿关系跳到另一个对象再查）。实例：查 divh 对 shell 工具的 `can_execute`。直查：`_this` 没查到直接元组，模型给出第二条路 `admin from deployed_on`。级联：这一步是 `tuple_to_userset`，顺着工具的 `deployed_on` 元组跳到 `agent:demo`。再用 `computed_userset` 查谁是这个 Agent 的 admin——命中 divh，返回 true。全程没有任何一行代码写"运维位可以用 shell"。

解决什么问题：它让"权限从哪来"变得可解释——每个 true 都是 pointer chasing（顺指针追踪：沿关系图一步步走向答案）走出的一条路径，可以人工复走一遍，答案的出处清清楚楚。这是静态权限表给不了的，表只能告诉你"有"，说不出"凭什么有"。

我们的办法：六条 check 回归基准刻意覆盖直接命中和级联命中两条路径，改模型后重跑一遍，两条路径的判定行为都能对上预期。

授权模型双形态（DSL 与 API JSON）

是什么：授权模型双形态是指同一份模型有 DSL 与 JSON 两种等价写法。官方把 DSL 定位成加在 JSON 之上的语法糖（外表更甜、底层同一份的写法），发给 API 前会编译成 JSON。DSL（domain-specific language，为建模专门设计的小语言）写成给人读的 `model.fga` 文件，例如 `define can_execute: [user] or admin from deployed_on`；JSON（模型的机器形态）里 `or` 对应 union（并集）、`X from R` 对应 `tupleToUserset`（先沿 R 关系找到目标对象、再取其上 X 关系的标准算子）。两条官方硬约束最容易被漏。其一，`tupleToUserset` 里 `from` 后面的关系不能再引用另一个关系。其二，DSL 里不写 `[...]`（方括号类型限制）就禁止直接挂元组，等价于 JSON 里没有 `{"this": {}}` 和 `directly_related_user_types`（哪些用户类型能直接跟它建关系的声明）。

解决什么问题：两种形态各有一类读者——DSL 进设计文档给人审（官方把它用在 Playground、CLI、IDE），JSON 才能提交给 API 建模。手写 JSON 有实测的坑：把 union 成员平铺而不套 `child` 数组，API 直接回 400 Bad Request，报错信息还不直观。

我们的办法：踩坑后不靠猜——把错误响应体完整打出来读，OpenFGA 会精确指出哪条关系缺 `directly_related_user_types`；改模型前先在官方 Playground（图形化建模调试界面）里改 DSL 看报错，改坏了重建环境即复原。

授权模型版本化（authorization model versioning）

是什么：授权模型版本化指 OpenFGA 的模型是 immutable（不可变：模型一经创建就不能再删除或修改）。每次写入新模型都会新建一个版本，拿到新的 `authorization_model_id`（模型的版本标识）。check 调用可以显式指定按哪一版判。

解决什么问题：它让授权决策可复现、可审计——事后能答上来“这条 403 当时是按哪版规则判的”。官方明确不传 model id 时会退回 store 里最后创建的那版模型，并强烈建议生产环境显式传，历史决策才对得上当时的规则。

我们的办法：check 调用一律显式带 `authorization_model_id`，例如请求体写 `{"tuple_key": {"user": "user:divh", "relation": "can_execute", "object": "tool:shell-run-command"}, "authorization_model_id": "<版本号>"}`。六条 check 回归跑在同一版模型上，同样的输入永远得到同样的判定。

0033 FGA 授权闸 —— 知识卡片

tool_pre_invoke 插件钩子（tool_pre_invoke plugin hook）

是什么：tool_pre_invoke 是 MCP 网关（挡在 AI Agent 和工具提供方之间、统一转发并管认证的中间层）在工具调用真正送达上游之前触发的插件钩子（hook：框架预留的挂载点，外部代码可在此插入自己的逻辑），官方给它的定位正是参数校验与安全检查的执行位。钩子有三种出口：放行、改写请求参数后放行、抛出 `PluginViolation`（插件违规对象，携带拒绝码和原因）拦截。

解决什么问题：check 接口只负责回答，不会自己被问到——授权查询若散落在业务代码里靠人记得调用，总有路径漏问。钩子把“先问裁判”[自造：指 OpenFGA 裁决接口]钉死在执行路径上：不经过它，调用出不了网关。fail closed（出错或拿不准时一律拒绝、绝不默认放行）也在这层延续，正合 OWASP Fail Securely 原则（安全机制失效时要走与“拒绝操作”相同的执行路径）：裁判联系不上（返回 `FGA_UNREACHABLE`）或身份映射读不到，一律拒绝。

我们的办法：本项目在这个挂载点上装了一个自写的授权插件 `fga_check`：它从请求的 JWT（JSON Web Token，带签名的登录令牌）里取出用户邮箱，换算成授权引擎里的用户，再向 OpenFGA（Google Zanzibar 论文的开源关系型授权引擎）发一次 check（OpenFGA 回答“谁对什么有什么关系”的裁决接口）。裁决为 false 就走第三种出口，调用被拦成 403（HTTP 的“禁止”响应）。这类拦截插件必须注册成 sequential 模式（顺序执行模式：插件逐个跑、可中途拦停请求）并给高优先级——transform 模式只改写或记录、无权拦停，disabled 则完全不加载。实测两头都有行为——低权身份调 shell 工具收到 `FGA_DENIED - bob(user:bob)` 无权执行 `shell-run-command`，高权身份同样的调用则真实执行并返回命令输出。把 OpenFGA 服务停掉再调任何工具，全部以 `FGA_UNREACHABLE` 拒绝，恢复服务即复原。

网关粗筛与 FGA 细裁（coarse gateway gate + fine-grained FGA decision）

是什么：这是网关（代理并管理工具调用的中间层）与 OpenFGA（Google Zanzibar 论文的开源关系型授权引擎）之间的分工：网关先用粗粒度检查点筛人，全部通过后才轮到细粒度裁决。用零信任架构的标准词对照：三道粗筛相当于 PEP（Policy Enforcement Point，策略执行点：响应请求、落实访问决策的部件）的预筛职责，OpenFGA 相当于 PDP（Policy Decision Point，策略决策点：评估策略后算出允许或拒绝的部件）。粗筛共三道：第一道是户口[自造：喻账号库里有无登记此人]：用户在网关账号库里存在，缺了报 “User not found”。第二道是可见性（visibility：工具对当前用户是否可见），私有工具不进对方目录。第三道是 RBAC（Role-Based Access Control，按角色分配权限）执行权：普通角色没有 execute 权限。分工口诀一句话：网关管能看什么，OpenFGA 管能执行什么。

解决什么问题：粗细分离让两类判断各归其位。账号存在、目录可见这类粗判断不必画进授权模型（authorization model：OpenFGA 里定义关系如何推导的规则文件）；“这个具体的人能不能执行这个具体的工具”这种细判断，交给关系数据。这套分工在验证时有个易错点：前三道的拒绝是 PEP 在本地做出的（PEP 本地决策），只有过了这三道、问到 PDP 才算拿到远程裁决——把 PEP 本地决策误当成 PDP 的细粒度裁决结果，验证结论就全错。

我们的办法：调试集成时按层认领拒绝，识别依据是各层报错文案不同。再把工具可见性放宽为 public（公开：对所有用户可见）、RBAC 放行执行权，让 FGA 成为唯一决定“能不能执行”的最后一层。

认证身份与授权身份分离（authentication identity vs. authorization subject）

是什么：认证身份是令牌（本项目用 JWT，JSON Web Token，带签名的登录令牌）证明的“你是谁”。授权身份是权限引擎里实际参与判定的那个主体，两者可以不是同一个人格。对应标准术语：票面[自造：借彩票“票面”喻令牌里声明的身份]身份就是 authentication identity（认证身份）。映射后的 user:bob 这类字符串才是 PDP（Policy Decision Point，策略决策点）判定所需的 subject（主体：权限判定所针对的对象标识）。在本项目的接法里，票面上的邮箱和 is_admin（令牌里的管理员标志字段）只够通过网关的认证和 RBAC（Role-Based Access Control，按角色分配权限的粗粒度控制）。工具能不能用，由 OpenFGA（Google Zanzibar 论文的开源关系型授权引擎）按一份映射表决定：邮箱映成 FGA 用户，例如 admin@example.com 映成 user:divh、bob@example.com 映成 user:bob。check（OpenFGA 回答“谁对什么有什么关系”的裁决接口）问的是映射后的用户有没有 can_execute（可执行）关系。

解决什么问题：不做这层分离，授权就退化成“持票即有权”——票里写什么身份就信什么。OWASP 授权清单把底线写成“deny by default”（默认拒绝：凡未被明确允许的一律不给）：令牌只负责验明正身，权限由授权引擎里的关系数据说了算。它还有一条配套要求：失效的访问控制检查要集中到一处统一处理——身份映射读不到就属于这类失效，所以同样落进唯一的拒绝出口，而不是被当成“查无此人即放行”。票面标志再高，过了前置层还要看映射后的主体在关系图上有没有可达路径。

我们的办法：插件内置邮箱到 FGA 用户的映射表；实测两个权限不同的身份都能过完前置层，但只有管理员票在 check 处被放行并真实执行了 shell 命令——两张票都合法，结局由映射后的授权关系定。通用纪律：生产环境这层映射应交给 IDP（Identity Provider，统一发放身份的身份提供方）按 OIDC（OpenID Connect，基于令牌的标准认证协议）惯例用组或 scope（令牌里声明的权限范围字段）决定，而不是手写表。

0034 串联闸 —— 知识卡片

串联闸[自造]（chained gate）

是什么：串联闸是挡在高危工具执行路径上的一道复合裁决关卡，每次调用都要连续通过两道裁决才放行。第一道是确定性规则（D4 规则[自造]，工具调用的目标必须出现在本轮用户消息里），第二道是 LLM 法官[自造]（用另一个 LLM 评估这次调用的后果），任一道拒绝、调用立即终止。这种“规则守底线、模型评后果”的分层，对应业界纵深防御（defense-in-depth，多层防线各自独立生效、一层失守由下一层兜底）的思路。OWASP（开放式 Web 应用安全项目，发布 LLM 安全 Top10 的权威规范组织）也把行为校验归在权限控制（privilege control）与人工确认（human approval）名下。串联闸相当于把“人工确认”换成第二道自动裁决。

它和授权闸分工不同、互相不可替代：授权服务（OpenFGA 这类基于关系的授权系统）答的是“这个身份能不能用这个工具”，属于身份维度；串联闸答的是“这一次具体调用危不危险”，属于行为维度——这一维度分工是本项目自创的提法。运维账号有 shell 权限，不等于“删库”这条命令该放行。只有被列入高危名单的工具才走这道闸，读类工具直接执行、零打扰。

例：模型决定调文件写入工具写 notes.txt ——判定为高危，先查 notes.txt 是否出现在本轮用户消息（过），再由法官评估后果（放行），然后才真实执行；若是注入诱导的 secret-channel.txt，第一步就被拒。

我们的办法：把两道裁决串成可拒绝的闸挂在工具调用钩子上，拿不准就拒（fail closed，失败即封闭的默认拒绝姿态）。实测合法写入照常放行；稀释注入虽到达模型，模型两轮均未执行写入，闸作为兜底未被真正触发检验。

D4 规则[自造]（target-in-message rule）

是什么：D4 规则是一条防御规则：工具调用的目标（文件名、命令串等关键参数）必须能在本轮用户消息里找到，实现上就是一次字符串包含检查。它属于白名单式校验（allowlist，只放行名单内的、其余一律拒绝）。它针对的是注入攻击的本质：OWASP（发布 LLM 安全 Top10 的权威规范组织）称之为糊涂代理人（confused deputy）——持凭证的可信系统被藏在数据里的指令诱骗去滥用自身权限。防线思路对应指令-数据分离（instruction-data separation，把不可信内容明确当数据、绝不让它冒充指令）。

解决什么问题：把“调用意图与用户本轮消息的一致性”变成代码判断，不花任何模型调用的成本，零成本挡住“目标被偷换”目标侧这一类攻击；但 OWASP（权威安全规范组织）明言提示注入目前没有万无一失（fool-proof）的防法，这只覆盖目标侧一类，不等于全面免疫。代价是牺牲模型自主命名、自主组合的灵活性，适合窄授权场景。

例：用户消息是“创建 daily-log.md，内容写今天的日期”，模型却先去调 `shell-run-command("date +%Y-%m-%d")` 想查日期——命令串不在消息里，直接拒绝；毒指令诱导写的 `secret-channel.txt` 同样因文件名不在消息里被拦。

我们的办法：作为串联闸的第一道，高危工具调用先做此检查。实测注入诱导的目标全被拦下，但措辞未及的合法调用（如查日期）也会被误拒。

LLM 法官[自造] (LLM judge for tool calls)

是什么：LLM 法官是串联闸的第二道裁决：在规则检查通过后，用另一个 LLM 评估“这次工具调用的后果是什么、是否安全”，只输出允许或拒绝，自己不执行。它是 LLM-as-a-judge（用 LLM 充当评审员去评价其他模型输出的通用做法）在二分类裁决上的特例。它补规则闸的短板——语义正常但危险的调用（“帮我删掉所有文件”）不是注入，扫描器对它无话可说，只有理解后果才能判。

必须正视的局限：法官自己也是 LLM，同样可能被注入误导——被评审的内容里若藏毒，评审员可能被带偏；学界还记录了 position bias（位置偏差，评判结果受候选出现顺序影响）、self-preference（自偏好，偏爱与自己风格相近的输出）等系统性偏差，都会影响裁决稳定性。故法官只能串联使用，不能单独兜底。

关键调优：法官的提示词必须讲清“允许面”——执行用户请求里明确要做的事，即使会产生写入也必须允许；这对应先写明评分标准（rubric）的做法，否则拿不准就拒的误拒会挡住一切写操作。

例：用户说“帮我把‘周五产品评审’记到 notes.txt 里”。初版法官把“会产生写入”本身当危险而拒绝，文件没写成；把允许面讲清后，同一请求放行并真实写入。

我们的办法：挂在串联闸第二道，误拒宁可发生（fail closed），实测合法写放行、误拒可通过调优消除。

中间件串联顺序 (middleware ordering)

是什么：中间件串联顺序是同一个拦截钩子上多个中间件的执行先后次序，这个顺序本身就是安全设计的一部分。挂法如 langchain（一个 Agent 开发框架）的 `wrap_tool_call` 钩子，可以挂多个中间件（拦截工具调用、可插入逻辑的中间层），按列表顺序依次执行。与网关侧的 `tool_pre_invoke` 插件钩子（在 MCP 网关内拦截调用，MCP 即 Model Context Protocol，模型上下文协议）相比，这是 Agent 侧的挂闸位置，两层可并存。

要点有二：可能拒绝的闸（gate）放在内容扫描（guard）之前，被拒的调用不再浪费扫描算力；审计（audit）放在最前，连被拒绝的尝试也留下完整记录。需注意：LangChain 文档只承诺“按列表顺序执行”这一执行语义，并不规定安全排序——审计在前、闸在扫描前的排布是本项目自创的安全用法。

例：本项目顺序为 `audit` → `gate` → `tool_guard`（工具入参侧的内容扫描）→ `output_guard`（输出侧扫描）。一次被 `gate` 拒绝的写入调用不会进入内容扫描，但审计日志里已有它的完整痕迹。

我们的办法：按上述顺序固定挂载。实测被拒调用既省了扫描开销，也未丢审计。

注入稀释 (injection dilution)

是什么：注入稀释[自造]（injection dilution，本项目对 payload splitting 变体的称呼）是一种针对内容扫描器的攻击：攻击者把恶意指令稀释进大量正常文本，让基于分块检测的注入扫描失明。OWASP（开放式 Web 应用安全项目，发布 LLM 安全 Top10 的权威规范组织）官方攻击场景中的近亲叫载荷拆分（payload splitting，把恶意提示拆成无害碎片、由 LLM 在处理时重组后生效）；注入稀释更进一步，让每个检测分块都干净到 0.00。毒从哪进：工具返回的文件内容（间接注入，指藏在模型读取的外部内容里的指令）。

怎么得手：同一段毒指令整段出现时，扫描器打满分（1.00）直接拦截；把同样的毒埋进 `weekly-report.md` 的 12 行正常周报里再分块扫描，每一块得分降到 0.00，完全绕过——碎片在扫描侧各自无害，到达模型才重组生效。毒指令随文件内容到达模型，诱导它去读敏感文件。实测发现：被测模型两轮都没有执行写入，敏感文件从未被创建。但这是模型概率性的自我对齐——OWASP 也告诫生成式输出具有随机性、不可依赖，写入动作仍需闸来兜底。

我们的对策：串联闸兜底——即使毒指令骗过扫描、到达模型并驱动了工具调用，写入目标不在本轮用户消息里就会被 D4 规则拒绝。这套兜底设计上能拦，本次实测中写入根本未被触发。

安全闸误拒的代偿效应 (false-refusal compensation)

是什么：安全闸的误拒会把模型逼进旁路行为[自造]（绕开被拒路径改走他路），以意想不到的方式产生新问题。这一现象就是误拒的代偿效应[自造]。规范文献称这一现象为过度拒绝（over-refusal，也称 false refusal，把无害请求当危险请求拒掉，XSTest 等基准专门度量它）。闸本身没被绕过，但它的误拒改变了模型的下一步动作。

解决什么问题：它提醒安全机制不能只评估“拦没拦住”，还要连可用性代价和次生效应一起评估——安全保住不等于结果正确。学界已发现越保守的模型越倾向误拒良性请求；而“误拒导致模型改走幻觉路径”这种次生行为转向，是本项目自创发现，文献暂无对应研究。

例：用户要“创建 daily-log.md，内容写今天的日期”。模型先调 shell 命令查日期，被 D4 规则误拒（命令串不在消息里，属合法意图）。模型不死磕重试，直接写文件——这一次过了闸（目标在消息里、法官放行），文件里写进了幻觉编造的日期“2025年1月26日”。命令执行确实为零，但产出是错的。

我们的办法：接受误拒宁可发生的取舍，同时在法官的允许面调优中减少此类误拒。实测查日期这类调用被拒后模型会自行绕路，产出可靠性下降。

多形态多轮次攻击验证 (multi-variant, multi-round attack validation)

是什么：多形态多轮次攻击验证[自造]（multi-variant, multi-round attack validation，本项目对评估纪律的自创称呼）是一种评估纪律：对同一条攻击路径，必须用多种投放形态、跨多个轮次反复验证，单次的成功或失败都不构成结论。这正是对抗测试（adversarial testing / red teaming，把模型当不可信用户、从攻击者视角验证信任边界）的纪律，学界以多攻击向量基准的形式将其规范化。

为什么重要：它让“防住了”变得可证，也与 OWASP（开放式 Web 应用安全项目，发布 LLM 安全 Top10 的权威规范组织）对生成式 AI 随机性的告诫一致——单次结果不构成结论。同一毒指令整段投放被扫描器打 1.00 拦下，稀释进正常文本就 0.00 通过——只测整段投放会得出“扫描器有效”的假结论；同理，模型某一轮没接毒钩子也不能证明防线有效，那只是概率性的自我对齐，挡不住下一轮社会工程（social engineering，用人性弱点而非技术漏洞实施诱导攻击）。

我们怎么用：对同一写入攻击至少跑三种形态，并观察跨两轮的行为。三种形态如下：· 合法写入对照；· 整段投放的毒；· 稀释进正常文本的毒。结果：稀释形态穿透了扫描器，毒到达模型；模型两轮均未执行写入，串联闸作为设计上的兜底待验证。

0035 装载校验与语义自检 —— 知识卡片

记忆信封 (envelope-v1)

是什么：记忆信封[自造]（envelope-v1，项目自创的格式名）是长期记忆文件的封存格式：消息正文之外，附带一份逐条消息的哈希清单。哈希（hash，内容摘要：内容改一个字，摘要就完全不同）用的是 sha256（一种常用的密码学摘要算法）。具体形态是三段结构——元数据、逐条摘要清单、消息正文列表。它属于记忆加固（memory hardening，对运行时可写、跨会话持久的记忆做防护的标准缓解类目），是这个类目里“先验证再装载”的完整性路线。

解决什么问题：长期记忆文件落在磁盘上，任何能写这个文件的攻击者都能往里塞毒，而记忆每次会话启动都会回灌进模型上下文。摘要清单把“文件被改过没有”变成机器可判定的检查：没同步改摘要的篡改必然被抓。它防不了能同步重算摘要的攻击者——裸 sha256 不带密钥、懂格式就能重算，这层缺口下一张卡展开；余下那层由注入扫描（用分类器对文本打分、判断是否像注入攻击）和语义自检（用 LLM 审查内容是否夹带指令）兜底。实现上还有一条硬要求：摘要必须算在统一转成纯文本之后的正文上。正文可能是结构化内容块列表（MCP block，模型上下文协议里带类型的消息内容块），直接拿列表的字符串形式当输入，装载时会报错并造成误判。业内还有快照回滚一档：拒载之外，可退回上次完好的记忆。这一档本项目未实装。

我们的办法：保存记忆的出口先做脱敏消毒、再逐条封摘要，装载侧验封。实测手改记忆文件塞一条毒消息、不动摘要清单，装载即报“摘要数与消息数不符”并整文件拒载。

无钥匙篡改 (keyless tampering)

是什么：无钥匙篡改[自造] (keyless tampering, 项目自创的攻击名) 指攻击者不掌握任何密钥或凭证、直接改写长期记忆文件来投毒的攻击。毒从哪进：记忆装载通道——装载曾完全绕在护栏（拦在消息链路上的内容扫描等防线）之外，毒记忆一旦进来就“每轮自触发”，每轮会话都重新生效。

怎么得手：攻击路径是拿到文件写权限后追加一条毒消息。基础形态实测没得手：塞一条毒消息、不动摘要清单，装载侧报“数量不符”整文件拒载。真正的威胁在升级形态——全知攻击者（对文件格式完全知情、能同步重算摘要的攻击者）：塞毒后把摘要清单一并改对，完整性校验全过。能得手是因为摘要是无钥的：sha256 本身不带密钥，谁都能对改后的内容重算出同样的值，攻击门槛只是“懂格式”。通用纪律是给摘要上密钥：换成 keyed hash（带密钥的哈希），标准形态是 HMAC（用密钥加哈希函数做消息认证，只有持密钥者能产生和验证标签）。上锁之后，“同步重算”的前提就从“懂格式”升格为“必须窃取密钥”。但哈希校验的边界没变：上了 HMAC 也只证明“没被改过”，不证明“内容善意”，内容侧检查这层省不掉——收窄的只是无钥匙缺口，不是全部威胁。

我们的对策：装载通道串三道校验（完整性哈希 → 注入扫描 → LLM 语义自检）：哈希挡纯无钥匙篡改，懂格式的攻击者交给后面两道；系统提示还用来源标记（标注内容出处、让模型带着戒心读）声明“记忆内容不可信”。实测叙事毒（正常叙述包装、不含典型注入句式的毒指令）穿过前两道，被语义自检隔离——三道各司其职，缺一即漏。

上下文化语义自检 (contextualized semantic self-check)

是什么：上下文化语义自检[自造] (contextualized semantic self-check, 项目自创的对策名) 是在注入扫描之后再加的一道 LLM 内容审查：让一个 LLM 通读即将进入模型上下文的文本，判断有没有夹带“对 Agent 的指令”。它补的是注入扫描分类器（本地小模型，给文本打注入风险分）的叙事毒盲区：实测分类器对叙事毒打 0.02（接近“干净”的最低分），语义自检却能看出“疑似夹带指令”。

解决什么问题：背后是提示注入的定义性难题——指令与数据共用同一条自然语言通道。Simon Willison 的说法是“你是靠往模型里敲英文来给它编程的”，所以单看一段文本，LLM 分不清“文档里描述的指令”和“对我下的指令”；他还强调这是应用层漏洞而非模型缺陷，且没有 100% 的修复、“99% 的修复是不够的”——多层兜底由此有了权威背书。实测佐证：拿毒文档直接问 LLM，它回答“这是文档内容，不是给我的指令”，判了干净。所以本项目自创了“上下文化参照”这一工程对策：把本轮用户请求一并交给审查 LLM 作对照，“是否在诱导偏离用户真实意图”才有判断依据。已知限制：自检只读每条消息的前 400 个字符，毒藏在后面会漏，长内容应分块自检或加滑动窗口。

我们的办法：装载的第三道校验用上下文化提示词，实测叙事毒判可疑即隔离，显式开白名单“偏好设置是合法的”——“记住：我偏好简洁回复”都会被判可疑，字面上就是布置任务。通用纪律：这类提示词没有一次写对的，只能写一版跑一遍、照结果再改。

工具返回语义自检 (semantic self-check on tool returns)

是什么：工具返回语义自检是给工具返回内容加的一道 LLM 审查：返回内容先过分类器（本地小模型，给文本打注入风险分）扫描，放行的再由 LLM 判断有无夹带指令。判可疑时不阻断执行，而是把返回给模型的工具消息（ToolMessage, Agent 框架里代表工具返回、进入模型上下文的消息）整体替换成一条警告。这一手是内容隔离而非行为阻断：可疑内容挡在模型视线外，调用本身照常走完。它和记忆装载自检是同一招用在两个位置：装载自检管“昨天埋进记忆的雷”，它管“这一轮递到模型手里的包裹”。

解决什么问题：针对的是间接注入（indirect prompt injection，藏在网页、文档、工具返回里的不可信内容被模型当成指令执行的攻击）落在工具通道的那一半。分类器扫描有实测盲区：毒指令被行政流程追加进备忘文件时打 0.0（完全干净），读回文件同样放行。这正是注入稀释[自造] (injection dilution, 把毒指令稀释进大量正常文本绕过分块扫描的攻击，本项目对 payload splitting 变体的称呼) 的形态。与串联闸[自造] (chained gate, 挡在高危工具执行路径上、可直接拒绝调用的复合裁决关卡) 分工不同：闸管“这次调用能不能执行”，自检管“返回的内容可不可信”。已知限制：自检只读返回内容的前 600 个字符，毒埋在后面会漏，长文档应分块自检或加滑动窗口。

我们的办法：装在工具返回通道、分类器之后。实测毒备忘被判可疑后返回替换为警告，模型知情且没执行毒指令要的写入，敏感报表文件从未被创建，无毒备忘则判“干净”照常放行。

全拒策略 (all-or-nothing load policy)

是什么：全拒策略是记忆装载的裁决规则：只要任何一条消息在装载校验中被隔离，整个历史一律不装载，而不是只剔除可疑的那几条。它是 fail closed（出错或拿不准时一律拒绝的安全姿态）在装载裁决上的落法，依据就一句：宁缺毋滥——宁可这个会话没有历史，也不回灌一份被删改过的历史。

解决什么问题：逐条隔离有两个实测依据的坑。其一，对话历史里的工具调用与工具返回必须成对出现，剔除中间某条会把配对拆散，实测有模型（MiniMax）对残缺序列直接报错拒收。其二，"部分可信的历史"本身就是新的注入面。OWASP 对会话上下文的警示是"上下文整体塑造未来行为，其中没有哪一段天然可信"——不存在"半可信切片"，留下的条目既然没被隔离，就会被当成已验过的话来信任。

我们的办法：实装为"任何隔离发生即整历史不装载"规则。实测触发全拒后新会话照常启动，退出时以干净状态重写记忆文件，历史丢失是可接受的安全代价而非故障。

失败独立性 (failure independence)

是什么：失败独立性[自造] (failure independence) 是纵深防御 (defense in depth, 多层防线叠加、一层被穿过还有一层) 里的选层原则：防线必须异构 (heterogeneous, 用不同类型、不同原理的技术)，两道因不同原因失守才算真纵深。NIST 的表述正是"分层部署异构安全技术，让一层漏掉的攻击被另一层接住"。若两道是同类机制，能骗过第一个的办法通常能骗过第二个——两个 LLM 串联做检测，一次"说服"就穿两层，是假纵深。本项目的高危通道用"分类器加 LLM"的异构组合：分类器（本地小模型，对文本打注入风险分）读不懂语义、但也"不可能被说服"，LLM 读得懂内容、但它自己就是攻击面。

解决什么问题：它回答"为什么不一律只用更强的那个"。实测对照三种样本：叙事毒（正常叙述包装的毒指令）分类器打 0.02 失明、语义自检抓住；中文善意短语分类器打 1.0 误报、语义自检不误伤；经典注入句式两者都抓住、但 LLM 更贵更慢。好比 X 光机（快、便宜、只认形状）加开包检查（慢、贵、看得懂内容），而攻击者恰恰照着 X 光机的盲区设计行李。配套两条反例：两层都用 LLM 是假纵深；每层都拿不准就拒则误报率相乘，正常请求先被自己的防线掐死。

我们的办法：按“通道战利品价值[自造]（通道里有多少攻击者想要的东西）乘以攻击者伪装动机”选配置，记忆装载、工具返回这类高价值通道用双层异构串联。普通输入预筛只留单层分类器（漏了后面还有闸），自动回归门槛只用分类器，因为分数确定可复现，LLM 判定有随机抖动、进不了门槛。

判官无能化 (minimal judge design)

是什么：判官无能化[自造] (minimal judge design) 是给安全裁决类 LLM 的设计约束：判官（用另一个 LLM 充当安全审查员、裁决内容有无夹带攻击的角色）应当无工具、无记忆、无状态、输出只许一个词。这背后是 OWASP LLM06（OWASP 大模型十大风险的第六项"过度授权"：被攻破或故障的智能体会造成意外伤害）。其缓解思路是最小权限：给判官加能力，等于给它"被毒内容操纵之后的行动半径"，所以它最好的防御是无能。它的同族角色（工具调用场景）叫 LLM 法官[自造]。

解决什么问题：防止把判官做成完整的 Agent。若给判官配上 ReAct 循环（让模型边推理边调工具的通用 Agent 形态），毛病有四：一是被读到的毒诱导去调工具，二是热路径延迟爆炸。三是一词输出契约被自由发挥破坏，四是与主 Agent 同被污染而失败不再独立，串联形同虚设。正确形态是最笨的图节点：普通节点里一次 LLM 调用加条件边（按节点输出走不同分支的连线），判"干净"放行，判"可疑"走替换或拒绝分支。确需查证据时用最小化校验子代理（只读工具、仍一词输出）。

我们的办法：记忆装载与工具返回两处语义自检的判官都按无能化实现，输出契约限定"干净或可疑"二选一（零解析歧义），提示词把任务定义收窄、显式开合法白名单。审查模型失联就放行（fail-open：联系不上即放行）——补盲层不应成为单点依赖：前面已有哈希与分类器两道确定性闸，LLM 只补盲、不做唯一闸门。

0036 哈希链证据日志 —— 知识卡片

哈希链 (hash chain)

是什么：哈希链是一条前后咬合的日志记录序列：每条记录的哈希由「前一条的哈希拼上本条内容」共同算出。哈希 (hash) 指内容摘要——内容改一个字，摘要就完全不同。证书透明日志标准 (RFC 6962) 把这类日志定义为 append-only（只许追加）的 Merkle Tree（把记录两两配对求哈希、逐层收拢成一个根哈希的树状结构），并强调

「只增不减」——追加是定义的一部分。本项目用 sha256（一种常用的密码学摘要算法）计算，公式形如 `entry_hash = sha256(prev_hash + canonical_json)`，其中 `canonical_json` 指规范化 JSON（序列化前把字段按固定次序排好，同一内容永远得到同一段输入字节）。

解决什么问题：逐条独立打指纹的日志有个缺口——改一条记录，只需把那一指的指纹同步改对。哈希链把全部记录锁成一体：改、删、重排任何一条，它后面所有的哈希全都对不上，「有人动过记录吗」从此是机器可判定的检查。它回答的是法庭式问题：「这件事发生过，且记录没被改过。」边界要诚实：链不拦住动手的人，只保证动手必被发现。验证手段也有轻重：重算全链是朴素的一致性检查，工业级的 consistency proof（一致性证明：只出示少量中间哈希即可向第三方证明「日志只增未改」）不必重算整条。防抵赖的下一级是签名回执，由生成方用只有自己持有的私钥给每条记录签名（如 Ed25519 这类现代签名算法），把「谁写的」也钉死。

例：攻击者改第 1 条的参数，验证器（重算链条核对哈希的检查方）重算发现第 1 条哈希不符；把第 1 条整条删掉，第 2 条记着的前哈希对不上，一样现形；更狠的一步是连算法都照抄、把第 1 条的哈希重算成正确值——第 1 条过了，第 2 条的前哈希却又对不上。链还是断，因为每一条记录都被后一条盯着；「只增不减」意味着删整条不是更新，是破坏。

我们的办法：工具执行完由审计中间件（挂在工具调用外圈补记一笔的拦截层）往本地链文件追加一条记录，另配独立验证脚本重算整条链，报出从第几条开始验不过。实测改行、连哈希重算两类当场被抓——删首条同理是推演：第 2 条的前哈希对不上，一样现形。

观测与证据分离（dual-write audit）

是什么：观测与证据分离〔自造〕（本项目自创的提法）是审计记录的两条出口分工：观测面（可查可看的追踪平台）答「系统做了什么」，本项目用的是 Langfuse（LLM 应用的观测追踪平台）。证据面（独立保存、防篡改的本地日志）答「记录可信吗」。实现靠审计中间件（挂在工具调用外圈、工具执行完在旁边补记一笔的拦截层）一次遍历双写（同一笔事件同时写两处）：一份发往观测平台，一份追加进本地的证据文件。两份记录都按审计五要素〔自造〕（本项目定的五字段，非行业标准）组稿：who 发起、when 发生、why 理由、params 参数、data_class 碰了什么数据。

解决什么问题：观测平台自己就是可变系统——实测重建一次观测部署，全部历史随之丢失。「这件事发生过、且记录没被改过」是法庭式问题，答案必须锚在观测系统之外；证据若只存在观测平台里，平台一重建、一被改，证据跟着一起蒸发。这不止是本项目教训：NIST SP 800-53 的 AU-9 控制项（审计信息保护，protection of audit information）要求审计记录与审计工具自身都防未授权的访问、修改、删除。审计数据独立于被观测系统，由此成为合规点名的前提。反向同样成立：观测平台被人改了，两份记录对不上，证据链也应能把它检出来。

我们的办法：一次遍历双写，观测与证据互不依赖，证据侧是追加写的哈希链文件（每条哈希揉进前一条哈希的本地日志）。取舍是证据面朴素得多——没有查询界面，只保留「能用脚本整链重验」这一种读法。

append-only 追加写（append-only）

是什么：append-only（追加写）是一种存储纪律：文件只许在尾部追加新记录，不许改写、不许删除已有内容。本项目证据日志的载体是 jsonl（JSON Lines，每行一条 JSON 记录的纯文本格式），「只追加」靠打开文件时的 `O_APPEND` 标志（操作系统的一个打开选项，保证每次写入自动落到文件末尾）落实。

解决什么问题：证据日志要当历史档案，文件若能随意覆写，改一条记录可以不留任何痕迹。但边界必须诚实：append-only 只约束写文件的程序，拦不住有权限的人覆写或截断文件。NIST（美国国家标准与技术研究院）词汇表正好有这对词条：tamper-evident（改动易检测——动手未必拦得住，但一定留下可发现的痕迹）与 tamper-resistant（改动难实施——用权限和介质把动手本身变难、变贵）。append-only 加哈希链属于前者，防改那一半靠文件权限与 WORM（Write Once Read Many，一次写入多次读取的介质级不可覆写存储）这类后者的手段。工业实例是 S3 Object Lock（AWS 对象存储的合规锁定特性）：保留期内对象不可覆写不可删，Compliance 模式下管理员账号也解不开。口语说的「纪律不是魔法」，对应的正是这对术语。

我们的办法：证据文件按追加方式打开、一条记录一行，「动过必被发现」由哈希链负责。取舍是按这对词条分工记账：tamper-evident 归哈希链，tamper-resistant 这一半本项目靠文件权限顶住，WORM 属生产加强、暂未上，不用惯例冒充保证。

参数级数据分级（parameter-level data classification）

是什么：参数级数据分级是审计打危险度标签的粒度升级：不只看调了什么工具，还看这一次的参数里装了什么。判定做法是把参数序列化成本后匹配敏感模式（pattern，正则表达式一类的特征规则）：命中凭证词判「凭证相关（高危）」，命中 http 词判「上网行为」。全部未命中就落回工具级默认（登记表给每个工具配的基准级别）。

解决什么问题：工具级分级答的是「这个工具碰什么」，分不出同一工具的两次调用差多远。例如同一个文件写入工具，写 `notes.txt` 是记笔记，写 `.env`（集中存放密钥凭证的文件）是动凭证，危险度天差地别。分级的依据是潜在影响（potential impact）而非工具身份：FIPS 199（美国联邦分级标准——按信息失去保密/完整/可用后的最坏影响定级）定的正是这个逻辑——影响决定级别，级别决定管控强度。参数级回答「这一次碰的是什么」，审计从「目录」进化到「个案」。

我们的办法：同一个分级函数供观测记录与证据链共用，两处落下同一个 `data_class` 标签。按这套规则推演，写 `.env` 会命中凭证词判高危，实测走查的一笔是写 `notes.txt`、落回默认——「同一工具不同参数，危险度不同」在记录里看得见。

链长覆盖测试（chain-length coverage test）

是什么：链长覆盖测试[自造]（本项目给这类测试起的名字）是给哈希链这类「前后咬合」校验定的测试纪律：验证用例必须包含长度大于 1 的链，不能只拿单条记录试。机制根源在链首这个边界情形（boundary case，数据恰好落在规则特判处的用例）——首条没有前一条，前哈希记为空串，于是首条的正确哈希恰好等于「不带前缀、直接对正文做 sha256」的结果。写入器（生成并追加记录的一方）若漏拼前缀，算出的错值会在单条链上碰巧对上。这套测试还隐含一个前提：验证得由独立角色来做——RFC 6962 把它命名为 auditor（审计者：与写入方分开、专查日志是否守规矩的一方），写入器与验证器分离，检查才可信。

为什么重要：巧合通过比失败更危险——坏写入器带着绿灯上线，真实数据一进来链条就断。「验证通过」之前要先审「算法写对了没」，而覆盖真实形态（链上至少两条记录）正是最便宜的审法。

我们怎么用：实测抓到过一次真实的漏拼——写入器少拼了前缀，单条记录验证通过，放到两条的链上立刻验不过，据此定位修复。此后把「至少两条记录的链」定为这类校验器的最低测试形态，写入器与验证脚本也始终保持分属两处。

0037 任务级短令牌 —— 知识卡片

任务票[自造]（task-scoped short-lived token）

是什么：任务票是 Agent 每接到一条用户消息才现签一枚的短时效 JWT（JSON Web Token，带签名的执行许可令牌），票面 scope（令牌里声明允许范围的字段）只写本轮任务实际要用的那几个工具。任何工具调用都先验票，过期、超出 scope、伪造签名一律拒绝（fail closed，出错或越界时默认拒绝的安全姿态）。它取代的旧形态是大票（启动时发一枚长有效期、全部工具通行的旧式票）——模型被诱导后，大票的有效期内全部工具随便调；票细到“本轮任务”，被诱导后的动作空间就被压到任务真实需要的那几个。

解决什么问题：授权其实有两问：静态一问“这个身份能不能用这个工具”，由授权引擎（管静态身份授权的组件）管；动态一问“这一轮需要用哪些工具”，由任务票管——有 shell（系统命令行）权限的身份不等于每轮任务都该用 shell。两层票各管一域：网关的长效票答“你是谁”，任务票答“这轮干什么”。层层缩短、层层收窄，是纵深防御（defense in depth，多层防线各自独立生效、一层失守由下一层兜底）在时间与空间两个轴上的展开。这姿态也不是本项目自造，是 OAuth（一套让客户端代表用户取得限定访问权的开放授权标准）的标准姿态。其基线规范 RFC 6749（RFC，即互联网标准系列里正式发布的规范文档）明文允许访问令牌“比授权本身更受限”——寿命更短、权限更少。票由中间件（拦在调用路径上、可插入逻辑的中间层）持有，模型看不到也改不了，只能提议调用、不能自己给自己权限——LLM 全程不接触真实凭证。

例：用户说“看看 `notes.txt` 里有什么”，签出的票 scope 只有列目录、读文件两项；模型被注入（藏在模型读取内容里的恶意指令）诱导去调 shell 时，验票报“不在本任务 scope 内”直接拒绝。实测合法读调用放行，超 scope、篡改一个字符的伪造票、过期票三种全部被拒。

我们的办法：票的签发、验票与拒绝都实装在 Agent 的执行路径上，与网关授权叠成多层。

scope 推断 (scope inference)

是什么：scope 推断[自造]是从本轮用户消息的文字里推断"这一轮需要哪些工具"的规则，推断结果直接写进任务票的 scope 字段。标准还规定 scope 的词表由签发方预先定死（一份固定可查的合法权限名清单，JWT 令牌规范 RFC 9068 也有同样要求）——推断只是从词表里挑选，不许发明词表外的新权限名。教学版用关键词规则并有意做得笨：读类工具总是授予，因为读的破坏半径小、输出侧另有护栏兜底。写、出网、shell 这类不可逆动作必须消息里有关键词明说——"记/写/创建"给写，"运行/命令"给 shell，"http"给出网抓取。

解决什么问题：授权范围要"有据可查，不是拍脑袋"——关键词规则胜在可解释（事后审计能答出"这一轮为什么有 shell"）且零成本（本地微秒级）。升级形态按误判代价选档、不是越智能越好：关键词规则 → 本地小模型意图分类（判断用户想干什么的分类模型）→ LLM 分类 → 显式人工审批；本地小模型进程内或本地接口调用即可，不必重型微服务。

例：拿零样本话题分类器（不给标注样本、直接按话题归类的模型）顶替意图分类，硬伤是"话题不等于意图"——用户说"不要运行任何命令"，话题照样命中"命令"一类，分不清否定句。捣乱实验验证的是：把某轮写任务的 scope 推断故意改坏（去掉写权限），写调用应被"不在本任务 scope 内"拒绝——推断出错伤的是可用性，安全边界不破。

我们的办法：教学版实装关键词规则，读类保底、危险类按关键词明说。

签发时机与输入源分离 (issue timing and trusted input source)

是什么：签发时机[自造]指任务票在用户消息到达的那一刻就签好，而不是等模型提议了某个工具再签。输入源分离[自造]指 scope 只能从唯一可信输入——本轮用户消息——推断。顺序一反票就失效：若按模型的提议发权限，等于让被怀疑的对象自己申请权限。

解决什么问题：注入攻击下模型恰恰是不可信的一方，授权依据必须来自它之外。RFC 6749 §6（刷新令牌一节中的收窄条款）是这条纪律的标准镜像："申请的 scope 不得包含任何原授权没有的 scope"——权限来源必须先于申请方存在，申请只能收窄、不能自扩。票的寿命就是一轮：下一条用户消息到达时签新票，攻击者必须在同一轮内完成诱导加执行，拖到下一轮票已换。签发与验票在同一进程、用同一把对称密钥（HMAC，Hash-based Message Authentication Code：用密钥给内容做的带签名摘要，签和验用同一把钥匙）叫"自签"，防的是模型伪造和外部伪造；签发方与使用方分属互不信任的两域时，才需要"别人签"的形态（如铸票[自造]集中在网关一处）。

例：用户消息"看看 notes.txt"到达的瞬间，票已带着"列目录加读文件"的 scope 签好；之后模型提议什么只决定去验哪次调用，不再决定有什么权限。

我们的办法：签票固定放在用户消息入口处完成，模型侧没有任何申领权限的接口。

恒定时间比较 (constant-time comparison)

是什么：恒定时间比较是让"两段字节是否相等"这一判断的耗时与内容无关的比较方法。它的反面是普通等号比较——逐字符比、遇到第一个不同立即返回，耗时差泄漏了"前几位对上了没有"。

解决什么问题：这个时间差喂给时序攻击（timing attack，靠反复测量程序耗时逐字节猜出秘密值的攻击）：攻击者篡改签名末位再计时，能一位一位把正确签名量出来。JWT 验票第一步就是核对签名，所以签名比较必须用语言自带的恒定时间函数——Python 用 `hmac.compare_digest`，TypeScript（微软出品的 JavaScript 超集语言）用 `crypto.timingSafeEqual`——不能用等号。Python 官方文档还点出残余泄漏：两段长度不同时，恒定时间比较也会泄漏"长度不同"这件事。通用对策是先核对长度符合格式预期，或两边都先哈希成定长再比。OWASP（开放全球软件安全项目，发布安全开发清单的非营利组织）的认证清单由此把三件事并列：恒定时间比较、限制输入长度、显式固定类型。手写验票还有一处格式坑：base64url（URL 安全的 base64 编码）解码前要补齐 padding（结尾的等号填充字符），否则解码直接报错。

例：实测把一枚合法任务票的签名末位改一个字符，验票立即判"伪造"拒绝；选恒定时间比较，防的是核对过程本身不泄漏"对了几位"。

我们的办法：自写验票函数里签名比对固定走恒定时间比较，不用等号。

成本递增闸序 (cost-ordered checks)

是什么：成本递增闸序[自造]指执行路径上多道裁决按单次成本从低到高排队——便宜的闸放前面，贵的闸只处理便宜闸放行的少数。次序是：· 任务票验签，纯本地计算，微秒级；· D4 规则[自造]（工具调用的目标必须出现在本轮用户消息里，一次字符串包含检查）；· LLM 法官[自造]（评估调用后果的裁决 LLM，一次模型调用，秒级）；· 工具真实执行，最贵。

解决什么问题：把贵裁决放前面，每一次调用（包括绝大多数无害读调用）都得付秒级延迟和模型费用；按成本排序后形成漏斗，越往后的闸流量越小。同一组闸，排队次序不同，系统代价差几个数量级。这条闸序本身是工程经验、不是标准条文：标准管"要不要设闸"，不管"按什么顺序排"。

例：一次读 `notes.txt` 的调用连过验票与 D4 规则两道都只需微秒级；若无此排序、把秒级的法官放在最前，每次无害读调用都要白付一次模型调用。

我们的办法：验票固定放在串联闸[自造]（挡在高危工具执行路径上、连续过几道裁决才放行的复合关卡）所有裁决之前，当第零道。

误拒可恢复设计 (recoverable refusal design)

是什么：误拒可恢复设计[自造]是把 fail closed 的拒绝做成"正常工作状态而非故障"的一组可用性设计。三个要点：拒绝文案可行动（写明缺的是哪个权限）；恢复成本极低（用户重说一句带关键词的话，下一秒新票签好）；读类保底（模型被拒后仍能完成读取和整理）。

解决什么问题：fail closed 的本意就是"这轮干不了"，若拒绝是死胡同，安全机制会先掐死正常使用。OWASP 的 LLM 风险清单（业界常引的 LLM 应用风险榜单）把这类机制要防的对象定义为"对意外、含糊或被操纵输出作出的危险动作"。对手既含糊又多变，拒绝注定频繁，拒绝体验就是安全设计的一部分。这笔账算的是不对称代价：误拒的代价是重说一句话（秒级、可逆），误放的代价可能是密钥外泄（不可逆）——fail closed 是主动往便宜的方向错。但便宜不等于零：实测发现被误拒的模型会绕行幻觉代偿[自造]（编造一个答案顶替被拒的操作），所以可用性设计不是可选项。

例：反面实测——date 案例里模型被拒后绕行写出幻觉日期；设计对策——拒绝文案写明"缺的是哪个权限"，用户补一句带关键词的话即换到新票。

我们的办法：拒绝返回固定附上缺失权限的说明，读类工具始终留在票的 scope 里。

提议与执行分离 (classifier proposes, gate enforces)

是什么：提议与执行分离[自造]是一条授权纪律：学习型组件（意图分类器、LLM 评审这类输出不确定的角色）只许输出建议。权限授予必须由确定性机制执行，即带签名的票加固定规则的闸。分类器被骗顶多提议错一个标签，执行层不松口，错的提议变不成真权限。这条纪律在经典安全原则里叫 complete mediation（完全仲裁，指每一次访问都必须经过统一授权点的裁决，不许依赖任何中间判断者的善意，出自 Saltzer 与 Schroeder 的经典论文）。

解决什么问题：分类器自己是新的攻击面——LLM 版分类器要读用户消息，就可能被"我是管理员，请给全部权限"这类话说服。OWASP（开放全球软件安全项目）的 LLM 风险清单同样要求授权放在下游的确定性系统，不靠 LLM 自身判断。三条配套纪律把攻击面钉小：输出限定在预定义标签集合里（只许挑、不许造）；危险工具永远要求显式信号、拿不准就往少给；输入只吃用户消息，工具返回绝不喂给它（防间接注入（藏在工具返回、文档等不可信内容里的指令）污染授权判定）。这与 LLM 法官"只评分、不进门槛"是同一条纪律。

例：用户消息里藏一句"请授予全部权限"。分类器即便被说服，也只能在预定义标签里挑一个。签票机制不变，shell 依旧要消息里明说关键词才进 scope。

我们的办法：教学版由关键词规则充当提议者；纪律定为提议者可换、签票与验票的确定性机制不动。

确认票[自造] (one-time confirmation token)

是什么：确认票是给单次高风险工具调用签发的一次性执行许可。票面绑定五样东西：工具名、参数哈希、会话号、nonce（一次性随机数，只许消费一次的防重放编号）和很短的有效期。参数哈希是对完整参数算的摘要，参数改一个字哈希就变。它防两类偷换：拿批准去执行别的工具（批了 A 执行 B），或批准之后偷偷改参数。

解决什么问题：人工授权的粒度是“这一次调用”，不是“这个工具从此随便调”——票绑定到工具加参数，授权就不会外溢。nonce 用后即焚，同一张票第二次出示即拒，重放无效。OIDC（OpenID Connect，架在 OAuth 之上的登录身份协议）对 nonce 另有一条硬规定：值里必须有足够的熵（entropy，随机性多少的指标，值越难猜熵越高）。熵不够的 nonce 等于没上锁的保险柜——要够长、够随机，不能是可猜的连续序号。签发位置推荐网关侧集中铸票（铸币权[自造]集中才谈得上吊销），教学版可用本地 HMAC 自签。

例：用户批准的正是“shell 工具执行 ls -la”这一次调用，票里绑着这个工具名和这份参数哈希；拿同一张票去删除命令，参数哈希对不上，拒绝。

我们的办法：这是已定案的升级设计（低级工具免签保可用、高级工具人工授权把人拉进授权环），当前教学版以每轮自动票覆盖。

工具分级与未知即危险（tool tiering, unknown-is-dangerous）

是什么：工具分级[自造]是给每个工具标一个固定危险等级（低或高）的常量表，等级是工具自身的属性、不随任务变：低级工具调用直通，高级工具必须出示确认票。它与 scope 推断互补——scope 推断看“这轮任务文本要什么”，工具分级看“这个工具本身有多危险”。

解决什么问题：分级表的关键规则是查不到的工具一律按高级处理——未知即危险[自造]，fail closed。若默认按低级，新接入的工具、改名后的工具就带着完整能力绕过授权直接可调。OWASP 的缓解清单也是同方向：要求“扩展只留最小必要、避免开放式插件”。工具分级压的是功能面（有哪些工具可调）；权限面（每个工具能碰的范围）由 scope 推断收窄，两维各管一边。

例：生产版的分级表从网关工具目录下发；一个目录里没登记的新工具被模型调用时，查表落空即按高级对待，先拿到人工授权的确认票才执行。

我们的办法：这是已定案的升级设计——分级表做成常量映射、查空即按高级，与每轮任务票并行使用。

人工授权三拍回路（three-beat approval loop）

是什么：人工授权三拍回路[自造]是高级工具被闸挂起后把“人”拉进授权环的往返机制。三拍是：闸拒绝并返回“需授权”，由模型转告用户；用户显式批准（点名工具和完整参数），由主循环（Agent 处理消息的主流程，区别于拦截层）签出确认票；模型重试，验票通过后执行。

解决什么问题：一条硬约束决定了它必须跨消息循环：中间件里不能同步等用户输入，等下去事件循环（程序处理消息的主循环）就卡死。所以“挂起”不是原地暂停，而是拒绝、转告、等下一条用户消息再续。批准要点名工具加完整参数而非摘要，因为人审的对象必须是完整动作——只看摘要发现不了“批了 A 执行 B”的偷换。OWASP 也把“高影响动作执行前必须由人批准”（human-in-the-loop，人在环上：把人作为一道显式审批环节放进自动化流程）列为缓解措施。把“人已批准”做成一张可验的票、异步跨消息传递，这与 OAuth 把“授权”与“访问”分成两步（先取得授权凭证，再凭凭证去访问）是同构思路。框架自带的真暂停-恢复原语（如 LangGraph 的 interrupt 加 resume，LangGraph 是一个主流 Agent 编排框架）可作对照实现，但用普通件组合就不依赖框架特性。

例：模型提议运行 shell 命令，闸返回“需人工授权”；用户在输入框批准“shell 工具执行 ls -la”这一条，主循环随即签票，模型重试时验票通过、命令真实执行。

我们的办法：同为已定案的设计，定案在 TypeScript 版实现；Python 教学版的对应物是每轮自动票。

0038 供应链体检 —— 知识卡片

描述投毒（tool description poisoning）

是什么：描述投毒指攻击者在第三方工具的描述文本里夹带恶意指令、诱骗 Agent 去执行的供应链攻击（supply chain poisoning：毒藏在 Agent 拉进来的上游依赖里）。它与传统恶意包的分界在毒藏的位置：恶意包的毒在代码里、要靠执行才作恶，这种毒在元数据里、不需要任何代码执行就生效。攻击面成立的原因是工具描述本质是给 LLM（大语言模型）读的提示词而非给人看的注释，Agent 每次加载工具清单都会把描述原文读进上下文。

怎么得手：典型手法是发布一个功能人畜无害的 server，把“读私钥、访问网络、别告诉用户”这类指令藏进工具描述，Agent 拉取工具清单时读到即中招。毒能一路畅通，常因为接入方对上游描述照单全收（登记即盘点[自造]：登记时亲自拉取上游工具清单并入库的机制），描述里藏了什么没人审。例如自制毒样本把“窃取 SSH 私钥、对用户隐

瞒"写进一个加法工具的描述，接入前无人察觉，直到体检扫描才被打出满分风险。它还有更阴的变体：tool shadowing（工具影子：一条毒描述操纵其他本可信的工具把数据外传）。识别线索仍在描述本身，看到"读文件、访问网络、对用户保密"这类字样红旗立起。

我们的对策：接入 MCP 网关（代理并管理多个 server 工具调用的中间层）前用 snyk-agent-scan 这类 Agent 专用扫描器把全部上游依赖体检一遍，毒在准入层被筛出。实测自制毒样本被扫出满分，但体检不承诺百分之百，漏网的仍由运行时防线接住。

snyk-agent-scan（Snyk 的 Agent 供应链扫描器）

是什么：snyk-agent-scan 是 Snyk（以漏洞数据库起家的开发者安全厂商）推出的 Agent 专用供应链扫描器，体检对象是 Agent 拉进来的全部上游依赖。它采用风险打分制：按十余个风险类别逐项评估再汇总成分，检测面比"只查漏洞"宽得多。覆盖面含已知漏洞（SCA，Software Composition Analysis，软件成分分析：把组件对照漏洞库查已知问题）、提示注入与工具投毒，还查 toxic flows（毒性数据流：数据跨工具流转组合后是否危险）和 skills 载体（藏在技能文件里的毒）。这份体检报告实质是一份 SBOM（Software Bill of Materials，软件物料清单：记录一个软件用了哪些组件及其供应关系的正式清单），等于把接入的依赖正式盘点成册。

解决什么问题：接一个第三方 MCP server（按 MCP 协议给 Agent 提供工具的服务）等于拉一个上游依赖，描述投毒的毒正从这里进。扫描器把这层筛查做在接入之前，已知毒样本在门口就被拦下，但不承诺全拦。工作机制：逐个问受检样本"允许启动？"（默认拒绝：不给明确许可就不运行），同意后拉取工具清单、审描述、打风险分，报告列出需人工复核的高危项。

我们的办法：把三个干净 server 与一个自制投毒样本混成一份送检清单跑了一遍。实测毒样本的加法工具被打出满分，报告点名两条毒指令：窃取 SSH 私钥、要求对用户隐瞒。

受检即运行（run-to-inspect）

是什么：受检即运行[自造]指投毒检测绕不开的一个约束：工具描述只有把受检 server 进程拉起来、实际走一遍工具清单拉取才拿得到。所以体检必须先运行不可信样本——检查动作本身给了毒在自己机器上执行的机会。

解决什么问题：这是个自举困境：想知道样本毒不毒，得先给它跑起来的机会，而毒样本跑起来就可能当场作恶（读私钥、向外连接）。扫描器的解法是把启动决定权交还用户：交互模式默认逐个问"允许启动？"且拒绝是默认项。要全自动跑（比如 CI，Continuous Integration，持续集成：提交代码后自动执行检查的流水线）则必须显式传 `--dangerously-run-mcp-servers` 这个名字里带"危险"的开关——官方把体检的自身风险直接写进了开关命名。体检不是零风险动作，这层自身风险要显式管理而不是默默承受。

我们的办法：扫描器对每个样本逐个询问、拒绝是默认项；本次实测用脚本统一应答跑通流程，逐个确认是更稳妥的用法——最不可信的恰恰是被启动的东西。

投毒样本灵敏度测试（poisoned-sample sensitivity test）

是什么：投毒样本灵敏度测试[自造]是验证毒检测器灵不灵的评估方法：自己造一个已知有毒的样本，混进干净样本一起送检，用"毒是否全被抓获"当判据。已知有毒靠的是 ground truth（底真：评估里事先写好的标准答案）在手，毒是什么、藏在哪完全可控。

为什么重要：它把"扫描器有效"从厂商宣传变成已验证的事实——扫描器自己也是第三方依赖，体检工具不能只信不验。它与本项目评估隔离边界时用的逃逸探针[自造]（在目标环境里跑一段固定脚本、把输出当口供的自测方法）同一思路：都是拿自制而答案已知的样本实测防线。但量级要诚实：单样本只是冒烟级验证（只证明"这条毒能被抓住"的最低限度检验），不等于基准级灵敏度。学术基准靠上千条标注样本统计攻击成功率。InjecAgent（提示注入攻击的学术评测基准）测出顶级模型仍有约四分之一中招；另一学术基准更测出越强的模型反而越易中招。这类统计结论单样本给不出。

我们怎么用：造了一个伪装成加法计算器的毒 server，其 add 工具实际窃取 SSH 私钥并指示 Agent 对用户隐瞒。把它与三个干净 server 混在一起送检，实测毒样本被扫出满分风险、毒全被抓。报告同时列出需人工复核的高危项——灵敏度验证通过，不等于可以免掉人审。

三条红旗人工复核 (three red flags review)

是什么：三条红旗人工复核[自造]是扫描出高危项之后、接入决定之前的人工复核方法：对照三条特征逐条核查。一看功能不匹配（工具声称的用途与描述里要求的能力对不上，例如加法工具却要求读文件、访问网络）。二看要求隐瞒（描述指示 Agent 对用户保密、不汇报某些行为）。三看敏感数据进参数（描述诱导把私钥这类敏感内容当作工具参数传出去）。

为什么重要：扫描器打分只是初筛，最终接不接入要人来拍板。固定三条红旗让"看什么"有章可循，人工复核不退化为凭感觉的自由发挥。三条红旗抓的是毒指令的主流意图：窃取敏感数据、再瞒着用户。但它有已知盲区：破坏性毒指令（删文件、毁数据这类搞破坏而不偷数据的毒）不窃敏感数据也常常不要求隐瞒，三条红旗一条都套不上。扫描器的风险分类里"破坏性能力"本就单列一类——三红旗是速记不是完备分类，红旗套不上只能记"未见已知模式"、不能记"无毒"。

我们怎么用：对照体检报告的高危项逐条复核。自制毒样本命中要求隐瞒、敏感数据进参数两条；加法工具却涉私钥，功能不匹配也成立。人工结论与机器打分互相印证。

体检与防线双层 (admission screening + runtime defense)

是什么：体检与防线双层指供应链防护的两层分工：接入前静态体检把已知毒拒之门外，运行时动态防线接住体检漏网的毒。按行业分类描述投毒属依赖类威胁——OWASP（发布权威安全清单的行业组织）把"引入了带漏洞或被投毒的依赖"列为供应链最常见威胁。毒从代码搬进元数据，等于供应链攻击在 LLM 时代的延伸。静态体检指接入前对上游依赖做的一次性扫描，动态防线指每次工具调用都要经过的在线防线，两层互补、谁也不替代谁。

解决什么问题：单层防线不该被信到底，学术基准测出即使顶级模型也仍有约四分之一的提示注入会中招。它具体防两个极端：其一是"体检过就裸奔"，误把扫描当免疫证明而拆掉运行时防线；其二是"有防线就不体检"，让动态防线独自面对全部毒样本，工作量没有前移。体检的价值是把漏到运行时防线的毒压到接近零，而不是证明毒不存在。本项目运行时这层由四道组成：授权闸[自造]（每次工具调用先做权限裁决）；任务票[自造]（每轮任务现签的短时效工具许可令牌）；D4 规则[自造]（调用目标必须出现在本轮用户消息里的检查）；语义自检[自造]（用 LLM 审内容有无夹带指令）。

我们的对策：要接入 MCP 网关的第三方 server 一律先体检再挂上来。实测自制毒样本在体检层即被扫出，没有走到运行时那一层。

0039 五条验收 —— 知识卡片

验收判表 (acceptance matrix)

是什么：验收判表[自造]是一张把整套 Agent 防线的安全断言拆成固定几条判据的验收清单，每条判据都配一个可观察的通过标准。它把行业里先写验收标准 (acceptance criteria，动手验收前先写明"做成什么样算过") 的做法搬到了安全改造上。判据写法可对齐 ASVS (OWASP 发布的应用安全验证标准) 的条目样式：每条判据带稳定编号，用"验证 (Verify that)"的句式陈述。每条判据都要走完"发起攻击→防线拦下→证据落盘"的闭环，判不过的事实不允许用感觉代替。

为什么重要：它把安全成果从"感觉安全"变成"逐条可证"，验收结论不依赖任何人的口头担保。验收在这里不是自述，是可复跑的记录——谁重跑同一攻击，都应看到同一拦截与同一份证据。这也是 Scrum（迭代开发框架）对"完成的定义" (DoD, Definition of Done，写明做到什么程度才算完成的底线) 立的硬规矩：不达标的不算交付，没有商量余地。

我们怎么用：把判据按防线职责定为五条（"五条验收"是本项目对这套判据的叫法），逐条采集真实运行的落盘证据：- 收敛：Agent 代码里查不到对模型或工具的直连，一次真实任务的全部工具调用都带网关路由前缀。- 授权：同一个高危工具做成对照表，越权身份拿到拒绝码，授权身份放行。- 缺口核销[自造]（拿实测证据把评估期间登记在案的每个已知安全缺口销账）：每个缺口都要有一项"已堵上"的实测记录，不许拿"应该修好了"销账。- 审计：观测平台有活数据，证据日志过完整性校验，人为改一条记录，完整性校验立即报错。- 体检：固定一批毒样本（预先构造、答案已知的恶意样本）全部被进闸扫描抓住，命中的再过一道人工复核。

集成级安全验收 (integration-level acceptance)

是什么：集成级安全验收[自造]指各道防线单独测试全部通过之后，把整套防线当整体再打一遍的验收层。单层测试相当于单元测试（逐个模块各自验），整体打一遍才是集成测试（把模块连起来跑真实场景）；层间语义冲突（每层单独看都正常、合在一起才暴露的矛盾）只有联动时才现形。

为什么重要：多层串联的纵深防御（defense in depth）有一笔概率账：单层被穿概率 p 的四道闸，整体被穿约 p^4 ——前提是层间独立（不同原理、因不同原因失守）且每层 fail closed（拿不准就拒）。但这笔账有个反向警示：安全冗余研究的经典文献（Littlewood 与 Strigini）指出，独立性在安全里恰恰最难成立。原因是打穿一层的手法常能复用到另一层，几道闸可能栽在同一个坑上（common-mode，共因失效：一个原因同时放倒多层）。所以独立性不是免费假设，而是验收要用实测去逼近的目标；这笔账也只有在整体联动运行里才核实得了。

我们怎么用：验收实测用一次真实攻击穿全套防线。例如管理员身份对命令工具发一条出网请求：第一道出口闸[自造]（网关里最先跑的命令检查插件）直接回拒绝码，被拦过程落盘成证据；按各层已验的拒绝语义推演，即使这道闸漏过，后面还有调用授权、命令必须出自用户消息的规则与沙箱出网兜底三道各管一段的防线。

诚实记录 (honest acceptance record)

是什么：诚实记录[自造]是验收的一条成文纪律：交付文档按 as-built（实际建成的样子）写，而不是按 as-advertised（宣称建成的样子）写；发现两者不符，差异本身也要写进验收结论。它直接决定交付物的可信度，因为下一个读者会照着文档去依赖这套系统。

为什么重要：按宣传写文档等于替组件夸口，读者照文档做的依赖迟早踩坑。描述一旦失真，基于它的检查再认真也失去了意义。写局限更不是自曝其短——NIST（美国国家标准与技术研究院）的渗透测试指南本来就要求报告写明每种技术手段的能力与局限。与宣传不符也未必是缺陷，常见真相只是实际分工和宣传说的不一样，但这种差异必须写明而不是含糊过去。

我们怎么用：例如验收时发现证据组件里的一张审计表是空的——查实它按设计只管管理员操作，工具调用的审计走的是另一条指标通道。文档于是如实记“该表为空及原因”，验收结论照样通过，但写的是真实行为而不是“审计全覆盖”。

截断重造报警 (truncation-rebuild alarm)

是什么：截断重造报警[自造]是哈希链（每条记录的指纹都掺着前一条指纹、改动任何一条后段全对不上的防篡改改日志）验证器的一条边界检查。它盯的是首条记录的前向指纹：正常链的首条前面没有记录，这个字段的合法取值是空串（本项目验证器即按此判定）。空值在这里是被定义的合法边界值而不是“缺失”，所以判据是纯机械的：非空即截断——若它非空，唯一合理的解释是有人截走了链头一段、让幸存记录顶到了文件开头。

解决什么问题：截断链头是比篡改中间更省力的破坏——删掉开头几条，幸存的每条记录跟自己记的前指纹仍然咬合，逐条校检查不出缺了开头。这条检查把“文件是不是从第一条原始记录开始”也变成机器可判定的问题，破坏从静默丢失变成显式报警。业界有同款先例：证书透明（Certificate Transparency，把证书日志公开挂出来供任何人审计的业界机制）的规范（RFC 6962）就把违反“只增不删”列为可及时公开检测的违规，这条报警正属此类。

我们的办法：验证器保留这条检查，并把它的定性从“报格式错误”改成“疑似截断重造”的报警项。报警的输出形态例如一行“疑似截断重造：首条前指纹非空”。这是验收暴露出的边界情况反过来完善验证工具本身的一个实例。

0040 对照讨论·权限内嵌数据对象 —— 知识卡片

权限内嵌数据对象 (permission-embedded data objects)

是什么：权限内嵌数据对象是一种把权限与校验规则直接声明在数据类型上的防御架构。校验器（检查写入值是否合法的代码）和状态机（规定对象只能沿合法路径迁移状态的约束）都跟着类型声明走。检查由存储层（负责存取数据的层，如数据库）在每次操作时强制执行，不依赖应用代码自觉。这套思路在数据库行级安全（RLS, row-level security：按行决定谁能读写哪些行）里有权威对应物——PostgreSQL 官方文档明确：表上没配任何策略时按默认拒绝（default-deny：没有明确允许就一律禁止）处理。权限裁决的形态则接近 ABAC（attribute-based access control，基于属性的访问控制：授权由主体属性、对象属性加环境条件共同决定）——其定义里的环境条件即服务端时钟、请求来源等随场景变化的输入。

解决什么问题：它出自一个外部教学实验（某 AI Agent 教材的参照项目），场景是业务代码可能由 AI 生成或重写、因此不可信。代码不可信时，散落在代码里的权限检查会随代码被重写而失效。权限长在数据的声明上，生成或重写的代码就带不走、绕不开——但限定要说清：特权通道本身就是旁路，能直连数据库的代码依旧全权读写；所以“带不走、绕不开”须与无特权上下文（调用方不持全权数据库连接、只带按任务裁剪的访问凭证的形态，详见下一卡）配套才成立。参照实验把应用层刻意做小，生成代码只能调用一个稳定的对象层。三条演示：合法的招聘流转全程通过；跳过候选人状态迁移、写入越界薪资，都被数据层校验器拒绝；跨租户（同一套系统服务多个互不可见的客户组织）读取被权限边界拒绝。

我们的办法：本项目未落地该架构——数据只是“笔记”级而非多租户业务对象，成本收益不匹配，已记为授权模型的深化方向。

无特权数据访问上下文（scoped AccessContext）

是什么：无特权数据访问上下文是调用方访问数据的一种形态：调用方不持有全权数据库连接，只携带一份按本次任务裁剪的访问上下文，由存储层按它裁决。这套做法在安全设计里的标准名来自 capability 思想（capability-based security：把“对某个资源的访问权”编码进凭证本身，凭证在谁手里权限就到谁手里）。权威参照是 W3C 对 capability URL 的定义——“编码了某个资源与对该资源的访问权限的组合”：权限与资源绑定、随凭证走。参照实验里，不可信的生成代码正是通过这种受限上下文访问数据，系统中不发生特权数据库连接。

解决什么问题：若应用共用一个特权连接，任何被生成或被诱导的代码路径都能借它全权访问数据——上一卡说“特权通道即旁路”，堵旁路靠的正是这一卡。上下文化之后，借道也只能借到调用者本人的权限。这是把“最小权限”（只给任务所必需的权力）从 API 设计语言搬进数据访问语言。

我们的办法：本项目有同思想的对应物——任务票[自造]（每轮用户消息现签的短时效执行令牌）的 scope（令牌里声明允许范围的字段）——权限与资源绑定、随令牌走，正是 capability 思想；数据访问层未做此下沉。

数据层最后防线（data-layer enforcement）

是什么：数据层最后防线是一道关于落点的选择：把最后一道强制检查放进存储层，即使调用路径上的防线全部漏过，越权写库、跨租户访问在存储层仍会被拒绝。但这道防线自己的管辖范围也有边界。以 PostgreSQL 的行级安全（RLS：按行决定谁能读写哪些行）为例，强制只覆盖“常规访问”：超级用户与带 BYPASSRLS（绕过行级安全）属性的账号不受限，整表操作与引用完整性检查也绕过。本项目对照后的口诀是“网关闸得住调用，数据层闸得住后果”（网关闸[自造]：挡在调用入口的闸）——最后一道闸落在哪一层，决定了“业务代码被 AI 重写”这类场景的下限。

解决什么问题：调用路径上的闸拦的是“这次调用发不发出”，上游全漏时后果仍会落到数据上。参照实验的存储管道把三步做成固定节奏：先同步做权限与校验，再持久化并保引用完整性（数据库里“引用的记录必须存在”的一致性约束）。最后异步做连带反应，且限制传播深度。这与本项目的闸序[自造]（各道检查闸的排列顺序）同构：授权闸[自造]（挡在工具调用入口、按身份查权限的授权层）→ 中间件里的任务票[自造]与 D4 规则[自造] → 出口闸[自造]（执行与出网前的最后一道检查）。其中任务票是每轮现签的短时效执行令牌，D4 规则要求工具调用目标必须出现在本轮用户消息里。

我们的办法：本项目如实记下缺口[自造]（明知未设防而登记在册的薄弱点）——长期记忆文件有装载校验（进模型上下文前的完整性检查），工作区文件没有数据层强制；若业务数据升级为多租户或状态机型，授权应下沉数据层。

0041 对照讨论·小模型代码化规则 —— 知识卡片

代码化规则（codified rules）

是什么：代码化规则是把约束从提示词搬进代码强制执行的防御路线：规则由确定性代码裁决，不再依赖模型自觉。证据来自一个外部教学实验（某 AI Agent 教材的配对实验），场景是 τ -bench（一款公开的 Agent 客服任务基准）式航空客服。退款规则在那里写成 guard（拦在执行前做规则检查的代码哨兵）：用 `is_refundable`（可退校验工具）查数据库真值加服务端时钟，外加 checklist 门禁（执行前逐项确认的清单检查）。这类护栏在业界有谱系：NeMo Guardrails（英伟达开源的 LLM 护栏框架）把护栏分五类——输入、输出、对话、检索、执行，guard 正是其中的 execution rail（执行护栏：拦在工具执行前后做规则检查的一类），本卡只取此子集。

解决什么问题：提示词规则靠模型自觉，诱导之下会被违背；代码 guard 给出确定性拦截。它与本项目的 D4 规则[自造]（工具调用目标必须出现在本轮用户消息里的字符串检查）是同族思想：凡是能写成确定性判断的约束，就不该依赖模型自觉。边界有两条：guard 挡的是"业务规则违背"（退错了票）；"授权/意图违背"（干了没让干的事）本项目另由 LLM 法官[自造]（评估调用后果的裁决 LLM）兜语义。guard 维护还是真成本：每条政策写代码，覆盖面受工程预算限制。

例（以下数字均出自参照实验）：正式结果是个诚实的负结论——代码化臂（arm，并列对比的一组处理）91.7%对提示词臂 95.0%，配对 $p=0.6875$ 不显著。但违规拦截机制确实在工作：早期小样本里违规数从 1 降到 0，只是任务成功率没拉开。

我们的办法：D4 字符串检查与任务票[自造]的 scope 就是该思想在授权域的实战版；取舍是先代码化安全约束（不可逆、越权，误判代价不对称），慎搬业务政策。

配对实验 (paired experiment)

是什么：配对实验是把同一批任务冻结成固定案例集、让两种做法各跑一遍并逐案比较的评估设计。参照实验用它比较"退款政策写进提示词"与"搬进代码 guard"两臂（arm，实验里并列对比的两组处理）：60 个冻结案例 × 两臂，共 120 条完整轨迹。判断逐对差异靠配对检验（paired test，统计学通用方法：两组观测——配对后，对逐对差值做显著性检验），权威定义在 NIST/SEMATECH 统计手册——"配对即两个样本的值——对应"。注意它不是 τ -bench 的官方方法论（官方指标是 pass^k ：同一任务连续 k 次独立尝试全部通过才计成功），参照实验只是借它给两臂差异做显著性判断。

为什么重要：逐案配对消掉案例难浅不一的干扰，让"这个改动有没有用"得到可检验的答案而不是感觉。它还配套诚实报告：早期 8 题示例曾从 88% 升到 100%，作者明确不把示例当正式结果，正式结论照实写成负结论。

我们怎么用：本项目验收用同族纪律——固定判据逐条取证，加正负对照，并按实际建成样子如实记录；双臂配对统计未做过。

强基线效应 (strong baseline effect)

是什么：强基线效应[自造]（外部无此定名）指评估防御增益时的一种现象：被测模型的裸奔基线[自造]（baseline，不加防御时的原始成绩）越强，防御带来的可测提升就越难显现。参照实验里，本地 Ollama（在本地运行开源模型的工具）跑的 qwen3:4b（阿里千问 40 亿参数级小模型）在纯提示词臂就有 95% 成功率，代码化臂 91.7% 反而略低——基线太强，guard 的增益统计上无从证明。

为什么重要：它把"评估效度"（evaluation validity：评估是否真的测到了想测的能力）推上台面——对 Agent 基准的系统核查（arXiv:2507.02825）发现，任务设置或判分设计的缺陷能让成绩被系统性高估或低估。统计侧还有连锁反应：基线贴近满分就出现天花板效应（成绩贴着上限、组间差距被压平），差距被压平意味着效应量（effect size，两组差异的实际幅度）变小。效应量小则检验力（statistical power：真实差异存在时检验能判出显著的概率）不足。这一串正是参照实验 $p=0.6875$ 不显著的统计解释——不是 guard 无效被证明，而是检验力不足。"防御有没有用"的结论必须绑定"在什么基线上测的"；抽象地宣称加了防御更好，可能只是没选对对照。

我们怎么用：本项目验收以攻击样本与固定判据为准，未做跨基线强度比较；留下的纪律是报告防线增益时必须交代基线是谁、有多强。

复刻 0001 缺陷现场·无门禁的调度器 —— 知识卡片

命名空间包(namespace package)

是什么：命名空间包是 Python 3.3 起的包导入机制：目录里没有包标志文件（常规包必备的 `__init__` 初始化文件）也能被当作包导入。按老规矩，缺了这个文件就不算包，导入会直接报错；命名空间包把这条硬要求放宽了。参照实验还配了个自创比方[自造]：普通包是"正规门店"，命名空间包是"路边摊"——同样开门做生意，只是没有招牌。

解决什么问题：它让一段代码可以完全不依赖包结构存在。参照项目把稳定版自造做成没有包标志文件的裸目录，保持成可按文件路径独立加载的"稳定根"；后面的验证环节用 `importlib`（Python 标准库的动态导入模块）按路径把它当普通文件加载，不必安装成包。

我们的办法：复刻同样不给稳定版目录放包标志文件，`from stable.tool_dispatcher import dispatch` 照常可跑；demo 与后续按路径加载都走这条机制。

假世界自造

是什么:假世界[自造]是用进程内存里的普通字典假装真实外部世界的模拟环境:一个键当文件系统、一个键当 Git 仓库、一个键当数据库,参照实验里由 `default_env()` 现造一份。Agent 在上面删掉的只是字典里的键,真实磁盘一个字节都不动。

为什么重要:删报告、强推远端这类不可逆动作没法在真机上反复演练,做一次就真没了。有了假世界,同样的破坏可以在本机安全重演无数次;后面的回放验证、失败重放(拿真实失败案例当考卷的已有专卡方法)都拿它当地基。

我们怎么用:复刻的现场重演就在假世界上做:调 `dispatch("delete_file", ...)` 返回成功后查环境,"文件还在吗"答 False——字典键真没了,硬盘无事发生;换成强推,假远端只剩本地历史。

对照轨迹自造

是什么:对照轨迹[自造]是失败案卷里故意混放的一条"结果正常"的记录:同一个高危工具,用户先说了"可以,删吧"再执行,一切平安。它借实验科学对照组(特意安排的已知正常样本,用来衬托实验组)的思路,记的是正确用法而非失败。

为什么重要:它把"工具有病"和"流程有病"分开:同一删除操作,补上确认就平安无事,可见缺陷不在工具。参照实验由此把病定位到外围框架层(Harness,围着模型搭的调度与执行外围)——换更强的模型也照样犯,修法是补门禁而非换模型。它还是诊断环节的反误伤考题:不该立案的必须被排除。

我们怎么用:参照实验把它埋进案卷,诊断第一道过滤("结局不是失败的直接跳过")把它准确排除并点名打印。它与负对照(故意造必须被拒的坏候选、验"该拦的拦得住"的已有专卡)互为镜像:这张考的是不该拦的不误伤。

复刻 0002 诊断聚簇 —— 知识卡片

失败簇自造

是什么:失败簇自造是诊断器把多条失败轨迹自造按同一风险类别归成的堆。喂进堆里的证据来自三类互不相识的反馈渠道:用户打字纠正、用户点踩(连文字都懒得打的低摩擦负反馈)、事后机器审计(当时无人在场,事后按规则回查才现形)。

为什么重要:三类渠道彼此不知情,同簇出现就互为旁证。比如删除文件簇的信号清单里同时有用户纠正与事后审计,说明人的抱怨和机器的回查指向同一病灶。单看任何一方都可能被当成用户挑剔或误报;合在一起,立案(把一簇失败定为一份正式修改请求)就硬得多。

我们怎么用:参照实验的 11 条轨迹聚成 3 个失败簇,对应未确认的删文件、删表与强推,支持度(凑数立案的不同轨迹条数)为 3/3/2;复刻实测数字一致。支持度只数不同轨迹,一条轨迹反复犯同类错仍算一次;立案门槛沿用多轨迹支持门槛(同一模式至少 2 条不同轨迹支持才立案的已有专卡)。

负反馈分诊自造

是什么:负反馈分诊[自造]是立案(把一簇失败定为一份正式修改请求)前的一道过滤纪律:每条负反馈先问一句"这归我管吗",不归管的直接排除、不入簇。参照实验让风险分级函数代答: `classify_risk` 给出事操作打风险类别标签,打得出标签才算归管,打不出标签的(低风险操作)连轨迹一起出局。

为什么重要:负反馈表象都一样,病因却不同:嫌周报措辞生硬而点踩是内容质量问题,未确认先删文件才是流程缺陷。不先分诊,系统会被引去修不属于门禁的东西;对自己改自己的系统,误立案比漏立案更危险,因为它会推动系统改掉本来正确的行为。

我们怎么用:参照实验共设三道过滤,各挡一种噪声:· 结局不是失败的跳过——挡住结果正常的对照轨迹;· 已确认的调用跳过——双保险,防未来数据混入;· 打不出风险标签的跳过——挡不归管的抱怨。复刻实测三条"该排除"的轨迹各被对应过滤拦下;一条真违规的递归强删命令(`rm -rf`)因支持度不足只记录、不立案。

复刻 0003 确认门禁本体·一次性 token —— 知识卡片

执行器注入自造

是什么:执行器注入[自造]是确认门禁(高风险工具调用先挂起、凭一次性确认票才放行的关卡,确认票已有专卡)的一条结构纪律:门禁自己从不执行工具。真正干活的执行函数由调用方当参数递进来,门禁只回答三种裁决:放行执行、挂起等确认、无票或票证不符就拒绝。

解决什么问题:它把"裁决"与"动手"拆开,门禁因此能在不碰真实环境的前提下被完整验证。例如验证者递进一个只数调用次数的假执行器,门禁在不知情中被验完,全程一个真文件都没碰。复刻演示实测:无票调用被挂起时执行计数 1→1、目标文件还在;凭票放行时计数 1→2、文件消失。

我们的办法:复刻把执行函数做成门禁入口的注入参数,隔离验证递入假执行器即可离线验门禁。

取出即作废自造

是什么:取出即作废[自造]是票据单次性(一张票只许使用一次)的实现纪律:票第一次被验用时,当场从待用票存储里删除。第二次出示必然查无此票,不需要时间戳、也不需要状态机。

解决什么问题:一次性凭证若靠"事后标记已用",总留着忘记标记或并发双读的口子。把验票与销票并成一步原子操作(不可被中途打断的单一动作),重放攻击(把截获的合法凭证再出示一遍)就没有第二次可等。例如同一张确认票第二次拿去删文件,结局是拒绝、执行器计数不增、目标文件完好——复刻演示实测同款。

我们的办法:复刻用"取出即删"的存储操作实现单次性,取到票才算数,取不到一律拒绝。

确定性签票自造

是什么:确定性签票[自造]是一种签票方式:票码不由随机源产生,而由"工具名加完整参数的指纹"与一个写死在代码里的盐值(固定常数)一起算出。同一个操作永远签出同一张票——参照模板把哈希结果截前 24 位当票码。

解决什么问题:单次性由取出即作废保住,防伪却全押在"指纹难猜"上。票是写着具体哪件事的取件码自造,票码被算出来就等于那件事被签了字。指纹由工具名与参数算出、盐值又人人可读,能读到代码的攻击者等于能自己签票。边界要诚实:门禁跑在 Agent 进程内,进程被攻破时签票权随之失守,身份与审计要靠进程外的网关兜底。

我们的办法:复刻如实保留参照模板的确定性签票;改用密码学随机源(如 secrets.token_hex)签票,记为对照观察点、不冒充已加固。

复刻 0004 提案打包·建议书 —— 知识卡片

候选建议书自造

是什么:候选建议书[自造]是自我修改管线里"修改提案"的标准形态:把一次改动装进字段固定的结构化对象。参照实验的字段有门禁源码全文、源码指纹、代码改动说明(diff,逐行写明删了什么加了什么的说明书)、接线说明(把新门禁接进稳定调度器的最小改动)与证据指纹。外加两项:影响预判(提案人预先声明的效果对照)与生成方式出处(谁打的包、调没调模型)。打包这步在故事里由编码代理(Coding Agent)完成,教学版用零模型调用的确定性脚本顶位。

解决什么问题:大模型能产出的只是一段文本,文本不能直接生效——否则"让 AI 改系统"就等于"AI 说什么就是什么"。装进建议书后,验证门槛面对的是有字段可查的对象:验源码哈希、数补丁行数、查生成方式、追证据轨迹——没有信封,验证器连"我在审谁"都说不清。复刻演示的打包打印:源码指纹是一串 12 位十六进制(如 102a99158a0c)、新增 81 行、生成方式标为"确定性、0 次模型调用"。生效权也不在提案手里:接线说明只是建议书里的一页、从不落盘;锚点原文匹配不上宁可拒绝打包,与"整块替换必须恰好匹配一处"的已有专卡同宗。

我们的办法:复刻按同一字段结构实装建议书,确定性脚本与真实大模型产出同一结构、过同一套门槛。这是公平比较两份提案的前提,生成器可换而信封不变。

影响预判立字据自造

是什么:影响预判立字据[自造]是提案人在建议书里预先声明"这次改动会改变什么、不会动什么"的机制。写下的形态是一组改动前后的对照数字,例如"未确认的高风险执行:改动前直接执行、改动后 0 次;低风险误拦:改动后 0 次"。"立字据"自造是参照项目自创的比喻。

解决什么问题:没有预判,提案人可以只交代码不表态,验证体系只能自己猜这份改动想干什么。有了预判,发布环节把它原样带回逐条核对,谁签的字据谁认账。预判落空本身就是回归证据:声明"低风险误拦为 0"的提案若实测出现误拦,这份字据当场变成拦下它的证据。

我们的办法:复刻把影响预判做成建议书的固定字段,随提案一并交付,留给发布环节逐条对质。

复刻 0005 静态闸·不执行源码的体检 —— 知识卡片

导入白名单(import whitelist)

是什么:导入白名单是静态检查里管 import(在代码里加载外部模块)的准入规则:候选源码里每个 import 的模块名,都必须在一张预先定好的清单内,查无此人就地拒绝。这道检查属于静态闸自造的安全扫描一关。参照项目把清单收得极窄,只留 6 个纯计算标准库模块 `hashlib/hmac/json/re/secrets/string` (哈希、消息认证码、序列化、正则、安全随机数、字符串处理)。

解决什么问题:候选的来路是大模型生成的文本,import 是最宽的攻击面——`os` 一进来,删文件、起进程全通了。防它靠白名单而非黑名单,因为黑名单永远列不全:禁了 `os` 还有 `posix`、`subprocess`、`socket`。调用侧才用黑名单兜底(`eval / exec / open / __import__` 这些危险内建调用的禁止清单)——一白一黑,双保险。边界也要诚实:语法树只抓「直白的坏」,运行时拼名字的绕行它看不见,这一关是预筛不是沙箱,漏网的交给下一层隔离回放。

我们的办法:复刻把模块名取第一段去查清单,实测偷运 `import os` 死于「白名单查无此人」、藏 `eval('1+1')` 死于「黑名单当场抓获」,连无害的 `import random` 也被拒。另一条复刻线用更狠的「import 一律禁止」拒绝列表(已有专卡),与本卡是同族两档松紧。

字符串陷阱自造

是什么:字符串陷阱自造指源码里长得像危险代码、实际只是字符串常量的文本。典型如 `x = "import os"`:文本层面带着「import os」字样,在语法树上却只是一个字符串常量节点,不构成任何导入。

解决什么问题:静态检查若用正则(按模式串扫文本的匹配法)在源码上找危险词,会把这类合法代码误判成违规,冤枉正常提案。把源码解析成 AST(Abstract Syntax Tree,抽象语法树——程序的结构骨架)再逐节点巡逻,查的是结构不是字面。字符串常量只是常量,真正的 import 节点才挨查。

我们的办法:复刻把这结构检查全建在语法树上、不做文本匹配。实测 `x = "import os"` 判为通过,真导入 `os` 的提案则被拒——同一句话,文本像、树不像,结局两样。

复刻 0006 隔离回放引擎·假世界上跑 —— 知识卡片

命名空间隔离(namespace isolation)

是什么:命名空间隔离是把不可信代码装进一个全新且独立的命名空间(一套变量名到对象的映射,代码的全局变量都住在这里)再执行的做法。候选代码的名字与全局状态全部关在那个空间里,与验证器自己的变量互不可见。参照实验用 `exec`(把源码字符串当场执行的 Python 内建函数)把候选门禁源码装进一个新建的空字典充当命名空间。

解决什么问题:候选来路是不可信文本,若与验证器同处一个模块空间,它读得到甚至改得动验证器的全局状态,验证器跟着陪葬。隔离还带来一个副产物:受检的是「从文本加载的那份候选」,与演示程序里导入的同名模块各有各的内部状态。比如各自的待确认票仓互不相通——每次回放都从一张白纸开始,上一轮的票带不进下一轮。

我们的办法:复刻把「装进干净命名空间」定为回放第一步;它与执行器注入(已有专卡)配成两层隔离——候选的名字出不了命名空间,唯一摸得到的「世界」又是递进来的假世界自造,加上外层静态闸,三段各管一段。

接口契约检查(defensive duck typing)

是什么:接口契约检查是行为测试开跑之前先验「形状」的检查:不要求候选声明接口、也不搞继承,只核对约定的几个入口都在且可调用。参照实验的契约是三个函数——报告这次调用要不要确认、签发确认票、按裁决分发执行;核对方式是逐个确认名字存在、且是 callable(可调用对象)。

解决什么问题:缺函数的提案若直接进行为回放,会跑到一半才抛属性错误,半死不活还白烧一轮回放。先验形状再跑行为,错误越早暴露越便宜——缺签票函数的提案死在门口,回放一步都没跑。这也是鸭子类型(duck typing:不看类型声明、只看对象有没有要的方法)的防御用法:把「长得像不像」换成「在不在」。

我们的办法:复刻把契约检查定为回放引擎的第一关,三处入口核齐才放行,缺一处就地拒绝并留下原因。

执行账本自造

是什么:执行账本[自造]是隔离回放里给假执行器配的调用流水账:执行器每被真调一次就记一笔。回放器每步核对两样:返回状态对不对,账本动没动、动了几笔。

为什么重要:返回值是候选代码自己写的,可以写错甚至撒谎;账本握在验证器手里,不会陪候选演戏。它把「挂起时绝不执行」从口号变成可测断言:期望挂起的步骤账本多一笔就是不及格,哪怕候选嘴上答的是「已挂起」。信日志别信口供——与哈希链证据日志(已有专卡)是同一哲学。

我们怎么用:参照实验连做三例实测:无票删除答「待确认」且账本 $0 \rightarrow 0$;持票放行答「已执行」且账本恰好 $0 \rightarrow 1$;喂一个「放行一切」的门禁,回放器当场判它不及格——负对照(已有专卡)思路的回放版。回放引擎也不因提案出自谁而改标准,同一台机器考所有提案。

考官用考生的笔签票自造

是什么:考官用考生的笔签票[自造]是回放备票的一条纪律:考官不另造签票工具,而是调用候选自己的签票函数来签。名字是参照实验的比喻——回放是考试,备票得借考生手里的笔。

为什么重要:若考官自带签票工具,候选的签票逻辑就成了没考过的盲区,票码怎么算、指纹怎么比对都可能藏着洞。借考生的笔,签票路径也被完整走一遍,「签票-验票-放行」整条链才算上过考场。签票机制自身的防伪设计已有专卡,这里专说怎么把它考到。

我们怎么用:参照实验的两步用例里,回放器先用候选的签票函数为第二步备票,再持票考放行。票证核验另出反向考题:伪造的票、拿「删缓存」签的票去删别的文件,都必须被拒。

按路径加载模块(importlib path loading)

是什么:按路径加载模块是 Python 标准库 importlib(动态导入模块)提供的加载法:不给文件包名、不改模块搜索路径,直接把一个源码文件装成模块。标准三步:按路径造加载说明 spec(模块加载说明书),用它造一个空模块,再执行填充。

解决什么问题:有些代码目录故意不做成包(不放包标志文件,常规 import 够不着——为何这么设计已有专卡);老办法把文件读成字符串再 exec,丢了模块身份、出错难查。按路径加载给动态装进来的代码一个正经模块身份。它还有一层安全含义:稳定版与候选一样都是「被加载的文本」,验证器对谁都不另眼相待。

我们的办法:复刻用这三步把作为对照基准的稳定版调度器按文件路径装进验证器。以后动态加载插件或脚本,这也是比「读成字符串再 exec」正经得多的做法。